IDauto

Dossier Technique et Architecture du Système

Audit du système implémenté — cas d'utilisation, classes, données, architecture logique et physique

Projet	IDauto — registre et carnet d'entretien de véhicules pour ateliers
Document	Dossier technique et architecture du système
Version du document	1.0 — 11 novembre 2026
Version du système décrite	branche `main`, commit `9a0f5b1` (en production sur idauto.tn depuis le 9 novembre 2026)
Sources	dossier `docs/architecture/` du dépôt `othoth77/idauto` (demande de fusion n° 48)
Vérifications sur les serveurs	lecture seule, 10 novembre 2026
Statut	document de revue — aucune modification de l'application

Document généré à partir des sources Markdown et Mermaid du dépôt. Les diagrammes sont des dessins vectoriels produits par Mermaid.

Table des matières

1. Présentation générale du projet 4
1.1 Objet du document 4
1.2 IDauto en bref 4
1.3 Légende des statuts 4
1.4 Conventions de lecture des diagrammes 4

2. Audit technique et technologies 6
2.1 Structure du dépôt 6
2.2 Technologies et versions 7
2.3 Inventaire des composants 7
2.4 État fonctionnel 8
2.5 Intégrations externes 10
2.6 Écarts avec les documents antérieurs 10

3. Diagramme de cas d'utilisation 11
3.1 Acteurs 11
3.2 Vue d'ensemble 12
3.3 Vues détaillées 13
3.4 Traçabilité : cas d'utilisation → route → droit 17
3.5 Fonctions absentes du diagramme 18

4. Diagramme de classes 19
4.1 Nature des classes 19
4.2 Vue A — entités du domaine 20
4.3 Règles métier invisibles sur le diagramme 25
4.4 Correspondance entité → table → code 25
4.5 Vue B — modules de service 26

5. Modèle de données 30
5.1 Principes 30
5.2 Inventaire des tables 30
5.3 Diagrammes entité-association 33
5.4 Relation décrite seulement par un commentaire 43
5.5 Index uniques partiels porteurs de règles métier 43

6. Architecture logique 44
6.1 Couches 51
6.2 Préoccupations transverses 51

7. Diagramme de séquence 52
7.1 Lecture 56

8. Architecture physique et déploiement 57
8.1 Production 59
8.2 Procédure de déploiement pratiquée 59

9. Sécurité, authentification et permissions 60
9.1 Authentification 60
9.2 Rôles et droits 60
9.3 Frontières de sécurité 60
9.4 Audit et données sensibles 61

10. Infrastructure, sauvegardes et environnements 62
10.1 Environnements 62
10.2 Sauvegardes 62
10.3 Observabilité 62

10.4 Configuration 62

11. Traçabilité entre diagrammes et code source 63

11.1 Diagrammes → fichiers sources → code 63

11.2 Traçabilité consolidée 65

11.3 Validation des sources 65

12. Limites, éléments partiels et fonctionnalités prévues 66

12.1 Éléments partiels 66

12.2 Fonctionnalités prévues 66

12.3 Fonctionnalités abandonnées 66

12.4 Non implémenté 66

12.5 Informations manquantes et hypothèses 66

13. Conclusion technique 67

Annexe A — Rendu des diagrammes et validation du document 68

1. Présentation générale du projet

1.1 Objet du document

Ce dossier décrit IDauto **tel qu'il est implémenté**. Il porte sur le code de la branche `main` au commit `9a0f5b1`, qui est la version en service sur idauto.tn depuis le 9 novembre 2026. Il s'appuie aussi sur des vérifications en lecture seule des serveurs de production et de pré-production, faites le 10 novembre 2026.

Il rassemble en un seul document les fichiers du dossier `docs/architecture/` :

- l'audit du projet ;
- les diagrammes de cas d'utilisation, de classes, d'architecture logique et physique ;
- le modèle de données.

Il n'ajoute aucune information technique qui ne figure pas dans ces sources. Il est purement descriptif : sa rédaction n'a modifié ni l'application ni sa configuration.

1.2 IDauto en bref

IDauto est **un seul processus Node.js 22, sans framework web**. Il s'appuie sur Better Auth pour les comptes et sur PostgreSQL 15 pour les données. Il sert, derrière nginx et sur un seul serveur privé virtuel OVH :

- des pages statiques en français et en arabe ;
- une API JSON.

- **Forme du système :** un monolithe en couches. Il n'y a ni tâche de fond, ni file de messages, ni cache, et aucun service externe n'est appelé pendant l'exécution.
- **Écritures :** chaque écriture est accompagnée d'une ligne du journal d'audit, dans la même transaction.
- **Rôles :** l'application web a trois rôles, technicien ⊂ gestionnaire ⊂ administrateur. Chacun est limité à sa propre organisation, et cette limite est appliquée jusque dans les requêtes SQL.
- **Données :** 41 tables dans le schéma `public` de la base `idauto_production`.

1.3 Légende des statuts

Le même vocabulaire est employé dans tout le document :

Statut	Signification
Implémenté	Le code existe, une route ou une page y mène, et il fonctionne en production.
Partiel	Une partie du code ou des tables existe, mais aucune page ne s'en sert, ou bien la fonction n'est pas configurée en production.
Prévu	Il n'existe que des documents, des fichiers README ou des tables vides ; aucun chemin de code.
Abandonné	Le code est encore présent mais désactivé (par exemple, la lecture automatique de la carte grise répond 410).

Ce que montrent les diagrammes.

- Une fonction prévue n'est jamais présentée comme implémentée.
- Les diagrammes de cas d'utilisation, de classes et d'architecture logique ne montrent que des éléments implémentés.
- Les quelques éléments partiels qui y figurent portent la mention « NOT CONFIGURED ».

1.4 Conventions de lecture des diagrammes

- **Libellés en anglais.** Les libellés des diagrammes sont repris tels quels des fichiers sources, qui sont en anglais. Ils restent ainsi identiques aux noms du code, ce qui permet de passer d'un diagramme au code

sans ambiguïté. Les légendes et les explications sont en français.

- **Dessins vectoriels en noir et blanc.** Chaque diagramme est dessiné en vecteurs, en noir et blanc, à partir de son fichier Mermaid. Sa forme et ses proportions sont conservées, sans recadrage.
- **Vues détaillées.** Un diagramme trop grand pour être lisible sur une page A4 est d'abord montré en **vue d'ensemble**, puis découpé en **vues détaillées**. Une vue détaillée reprend à l'identique une partie des éléments du diagramme, avec toutes leurs relations.
- **Pointillés et boîtes vides.** Dans une vue détaillée, un élément dessiné en **pointillés** (pour les schémas en blocs), ou une classe ou une table sans attributs, est détaillé dans une autre vue.
- **Complétude des vues.** Un contrôle automatique vérifie que l'ensemble des vues détaillées contient chaque élément et chaque relation du diagramme d'origine (voir l'annexe A).
- **Tables et classes.** Les tables de la base ne sont pas des classes de l'application. Les 41 tables sont décrites au chapitre « Modèle de données ». Le diagramme de classes montre les 15 entités du domaine manipulées par l'application et indique, pour chacune, la table qui la porte.

2. Audit technique et technologies

L'audit a été réalisé le 10 novembre 2026 sur `main` au commit `9a0f5b1`. Les sources sont :

- la lecture du code source ;
- l'extraction du schéma et des routes par script ;
- des vérifications en lecture seule sur les serveurs.

Rien n'est décrit à partir d'une intention. En cas de désaccord entre un document ancien et le code, c'est le code qui fait foi.

2.1 Structure du dépôt

Chemin	Contenu	Exécuté en production
`reference/`	L'application : un serveur HTTP Node.js (`api.js`), ses modules de routes, services et dépôts, les pages statiques (`*.html`) et leurs scripts (`app-*.js`, `admin-*.js`)	**oui**
`reference/workspace/`	Services de l'application simplifiée : enregistrement d'un véhicule, fiche véhicule, contacts propriétaires, interventions, listes et tableau de bord	oui
`reference/auth/`	Configuration de Better Auth (`auth.js`) et passage de la session au principal (`principal.js`)	oui
`reference/documents/`	Photos de carte grise : stockage, métadonnées, analyseur OCR retiré	oui (les routes OCR répondent 410)
`reference/vehicle/`	Normalisation des plaques, validation du VIN, dépôt et résolution des véhicules	oui
`reference/workshop/`	Dépôt des visites, opérations et commandes d'atelier ; service de lecture de plaque de l'atelier	oui
`reference/parts/`	Catalogue local de pièces, adaptateur TecDoc (non configuré), fournisseur fictif (tests uniquement)	oui, catalogue local seulement
`reference/admin/`	Modèles de lecture de la console d'exploitation	oui
`web/citizen/`, `web/design-system/`, `web/vendor/`	Pages publiques, système de design partagé (variables, CSS, petits scripts), bibliothèques navigateur embarquées (Tesseract, scanic, qrcodegen)	oui
`database/`	`schema.sql`, migrations ordonnées, jeu de données de test fictif	oui (appliqué)
`ops/`	Outils d'exploitation : comptes (`auth-users.js`), sauvegarde hors serveur (`offhost-backup.js` et `adapters/s3-compatible.js`), médias, générateurs de pages et de catalogue, procédures	lancés par l'opérateur, ce ne sont pas des services
`config/idauto.example.json`	Formats de plaque, interrupteur de la résolution publique et limite de débit	lu à l'exécution par `plate-validator.js`. L'interrupteur public est lu dans le fichier désigné par `IDAUTO_PUBLIC_RESOLUTION_CONFIG_PATH` ; en production, c'est un fichier `idauto.json` local au serveur, avec `enabled` = true
`tests/`	44 suites : 16 sans base de données, les autres sur une base PostgreSQL jetable	intégration continue et poste local
`protocol/`	Schémas JSON et vocabulaires du protocole ouvert d'identité des véhicules	non (spécification)
`src/`	Fichiers README seulement (`api`, `blockchain`, `identity`, `provenance`, `verification`)	non (prévu)
`docs/`	Documents produit, confidentialité, sécurité, déploiement et historique	—

2.2 Technologies et versions

Couche	Technologie	Version	Où elle a été constatée
Exécution	Node.js	22.22.1 (production et pré-production)	`node --version` sur les serveurs ; l'intégration continue utilise Node 22
Serveur HTTP	module `http` de Node, sans framework web	—	`reference/api.js`, `createServer()`
Authentification	Better Auth (courriel et mot de passe, sessions par cookie)	1.7.2	`package-lock.json`, `reference/auth/auth.js`
Pilote de base de données	node-postgres (`pg`)	8.23.0	`package-lock.json`, `reference/db.js`
Base de données	PostgreSQL	**15.18** en production (conteneur Docker) ; 16.9 en pré-production	`SELECT version()` ; binaire de pré-production
Interface	HTML statique et JavaScript sans framework, variables CSS de design	—	`reference/*.html`, `web/design-system/`
Bibliothèques navigateur	Tesseract (OCR du lecteur de plaque de l'ancienne page atelier), scanic, qrcodegen	embarquées	`web/vendor/`
Proxy inverse et TLS	nginx, certificats Let's Encrypt (Certbot)	nginx 1.28.3	`nginx -v` ; émetteur du certificat
Conteneurs	Docker (pour PostgreSQL seulement)	29.6.1	`docker --version`
Système (production)	Ubuntu	26.04 LTS	`lsb_release`
Intégration continue	GitHub Actions, `ubuntu-latest`	—	`.github/workflows/ci.yml`
Dépendances d'exécution	deux exactement : `better-auth` et `pg`	—	`package.json`

2.3 Inventaire des composants

Composant	Nature	Responsabilité	Source
Serveur HTTP et chaîne de traitement	module	Fichiers statiques avec CSP, contrôle d'accès aux pages, route publique, routes d'authentification, authentification, contrôle des droits, aiguillage	`reference/api.js`
Routes de l'espace de travail	module de routes	API de l'application simplifiée : enregistrement, fiche, propriétaire, détails, suppression, interventions, pièces, tableau de bord, recherche, notifications	`workspace-routes.js`
Routes V12	module de routes	Pages atelier, identification par plaque, visites, opérations, commandes, catalogue	`v12-routes.js`
Routes V14	module de routes	Photos de carte grise ; routes OCR retirées (410 sauf si `IDAUTO_CG_OCR_ENABLED=1`)	`v14-routes.js`
Routes de la console	module de routes	Modèles de lecture d'administration : tableau de bord, véhicules, entretien, organisations, sources, qualité, audit, comptes	`admin-routes.js`
Service d'enregistrement	service	Valider et enregistrer un véhicule (plaque obligatoire), contrôle des doublons, contact propriétaire, note	`workspace/registration-service.js`
Service des interventions	service	Saisir, modifier ou annuler une intervention avec ses pièces et leur provenance déclarée (deux valeurs)	`workspace/intervention-service.js`
Détails du véhicule	service	Modifier marque, modèle, année, kilométrage et note propre à l'atelier ; supprimer = archiver	`workspace/vehicle-details.js`
Contacts propriétaires	service	Contact du propriétaire, privé et propre à chaque atelier ; chaque lecture est auditée	`workspace/owner-contacts.js`

Composant	Nature	Responsabilité	Source
Dépôt de l'espace de travail	modèle de lecture	Fiche, listes, compteurs du tableau de bord, recherche globale, cloisonnement par organisation	`workspace/workspace-repository.js`
Service des documents	service	Enregistrer, lister et supprimer les photos de carte grise (adressage par contenu), vignettes	`documents/document-service.js`
Dépôt et résolution des véhicules	dépôt / service	Plaques, faits VIN, historique d'identification, résolution	`vehicle/*.js`
Dépôt de l'atelier	dépôt	Visites, opérations, commandes (V12)	`workshop/workshop-repository.js`
Écritures et audit	accès aux données	Chaque écriture est jumelée à une ligne `idauto_audit_log` dans une même transaction (`withAudit`)	`reference/writes.js`
Réserve de connexions	accès aux données	Une réserve `pg` unique, configurée par `IDAUTO_DB_*`	`reference/db.js`
Stockage des médias	accès aux données	Fichiers adressés par contenu sous `IDAUTO_MEDIA_STORAGE_PATH`	`reference/storage.js`, `documents/document-storage.js`
Authentification	sécurité	Better Auth (connexion limitée à 5 par minute et par client), passage au principal, identités de service par jeton, session navigateur du propriétaire	`auth/auth.js`, `auth/principal.js`, `identity.js`, `session.js`
Limitation de débit	sécurité	Fenêtres par identité pour la résolution publique, la recherche et l'enregistrement	`reference/rate-limit.js`
Observabilité	transverse	Une ligne JSON par décision d'authentification ou par événement, sur la sortie standard	`reference/observability.js`
Internationalisation	présentation	Source en français, catalogue arabe, `lang` et `dir` fixés côté serveur	`i18n.js`, `i18n-serve.js`, `i18n-catalog-ar.js`
Outil de gestion des comptes	outil d'exploitation	Créer, supprimer, changer le rôle, définir le mot de passe, révoquer les sessions	`ops/auth-users.js`
Outil de sauvegarde hors serveur	outil d'exploitation	Préparer, inventorier, vérifier, envoyer vers un stockage compatible S3, vérifier une restauration	`ops/offhost-backup.js`, `ops/adapters/s3-compatible.js`

Tâches de fond. L'application ne démarre **aucun planificateur, processus de fond ni minuterie**. La sauvegarde planifiée tourne sur le serveur de production sous forme de minuteries systemd. Ces minuteries sont installées depuis un autre dépôt (`othoth77/mythos-prod`) ; voir le chapitre « Infrastructure, sauvegardes et environnements ».

2.4 État fonctionnel

Fonction	Statut	Élément de preuve
Connexion, déconnexion, changement de son mot de passe, sessions (Better Auth)	**Implémenté**	`auth/auth.js`, `/api/auth/*`, tests `ida-v13-*`
Réinitialisation du mot de passe	**Implémenté comme une étape d'exploitation** : le bouton « Mot de passe oublié » affiche un message, puis l'opérateur lance `ops/auth-users.js set-password` ; aucun courriel	`login-ui.js`, `ops/auth-users.js`
Création et suppression de compte	**Outil d'exploitation seulement** ; la page web « Gestion des utilisateurs » affiche les comptes en lecture seule	`ops/auth-users.js`, `/api/admin/accounts`
Page des rôles et des droits	**Implémenté** (matrice en lecture seule issue de `principal.js`)	`/api/admin/roles`

Fonction	Statut	Élément de preuve
Français et arabe, écriture de droite à gauche	**Implémenté**	`i18n*.js`, tests `ida-ui-browser`
Liste, recherche, enregistrement (seule la plaque est obligatoire), consultation et modification des véhicules	**Implémenté**	`workspace-routes.js`, `registration-service.js`, `vehicle-details.js`
Suppression d'un véhicule	**Implémenté comme un archivage** : `fiche_status = 'archived'`, plaques annulées, fait VIN désactivé. La suppression est refusée s'il existe une intervention.	`vehicle-details.js` `remove()`
Contact du propriétaire (privé, propre à l'atelier, audité)	**Implémenté**	`owner-contacts.js`
Photos du recto et du verso de la carte grise	**Implémenté** (stockage privé, route authentifiée, contrôle de l'organisation)	`v14-routes.js`, `document-service.js`
Lecture automatique (OCR) de la carte grise	**Abandonné** : les routes répondent 410 sauf si `IDAUTO_CG_OCR_ENABLED=1` ; l'interface a été retirée	`v14-routes.js` `ocrRetired()`
Interventions avec plusieurs pièces et provenance à deux valeurs	**Implémenté**	`intervention-service.js`
Liste des pièces (filtre par provenance, recherche)	**Implémenté**	`listPartsUsed()`
Tableau de bord (5 compteurs réels)	**Implémenté**	`overview()`
Recherche globale	**Implémenté**	`/api/search`
Passeport public par IVID	**Implémenté** et activé en production (configuration du serveur `idauto.json` : `enabled` = true ; par IVID uniquement, débit limité). La recherche publique par plaque est fermée (404).	`api.js` `handlePublicPassportRoute`
Réception d'observations, faits, décisions de la file de revue, fusion et séparation de véhicules	**Implémenté** (API et console d'exploitation, hors du menu simplifié)	table `ROUTES` de `api.js`, `admin-routes.js`
Ancienne page atelier avec lecture de plaque dans le navigateur (`/atelier`)	**Implémenté**, hors du menu simplifié	`atelier.html`, `v12-routes.js`
Commandes et stock d'atelier (`idauto_workshop_orders`, `idauto_org_stock`)	**Partiel** : les tables et les fonctions de dépôt existent, mais aucune page de l'application simplifiée ne les utilise	`workshop-repository.js`, `local-parts-catalog.js`
Catalogue de pièces TecDoc	**Partiel, non configuré** : l'adaptateur est présent, mais aucune variable `IDAUTO_TECDOC_*` n'existe sur le serveur de production	`parts/tecdoc-adapter.js`
Fournisseur sous licence de données véhicule (plaque ou VIN → véhicule)	**Partiel, non configuré** : adaptateur HTTPS générique, désactivé tant que `IDAUTO_VEHICLE_PROVIDER_URL` n'est pas défini. Aucune variable `IDAUTO_VEHICLE_PROVIDER_*` en production : la résolution utilise uniquement la base locale.	`vehicle/providers/http-vehicle-provider.js`, `vehicle/vehicle-resolver.js`
Caméras et lecture automatique de plaques, mouvements de véhicules, consentements, vérifications, scans de documents, table des rôles, localisations d'observation, table de la file de revue	**Prévu** : les tables existent dans le schéma (en production aussi), mais aucun chemin de code ne les lit ni ne les écrit	chapitre « Modèle de données », statut « schéma seulement »
Ancrage blockchain, service de provenance, schémas de la plateforme Mythos (`mythos_core`, `atelier_network`, `fixpert`)	**Prévu** (documents et README seulement)	`src/*/README.md`, `docs/BLOCKCHAIN_ARCHITECTURE.md`, `docs/ARCHITECTURE.md` §2
Intégration Piece.Autos ou Casse.Autos, facturation, gestion de stock	**Non implémenté** (exclu par le propriétaire du projet)	`docs/SIMPLIFICATION_2026-11.md` §6

2.5 Intégrations externes

Intégration	Statut	Sens
Better Auth	bibliothèque intégrée au processus (aucun appel externe)	—
Stockage hors serveur compatible S3	utilisé par l'outil de sauvegarde (opérateur ou minuterie), pas par l'API	HTTPS sortant depuis le serveur
TecDoc	adaptateur seulement, non configuré	(serait du HTTPS sortant)
Fournisseur sous licence de données véhicule (recherche par plaque ou VIN)	adaptateur seulement, non configuré ; aucun fournisseur choisi	(serait du HTTPS sortant, jeton porteur lu dans l'environnement)
Let's Encrypt	émission des certificats par Certbot sur le serveur	sortant
Courriel, SMS, paiement, mesure d'audience	**aucune**	—

2.6 Écarts avec les documents antérieurs

Document	Affirmation	Réalité au 10 novembre 2026
`docs/ARCHITECTURE.md` (11 août 2026)	aucun service d'API ni d'interface IDauto n'est déployé	l'API fonctionne sur idauto.tn derrière nginx depuis novembre 2026
`docs/ARCHITECTURE.md` §2	schémas cibles `mythos_core`, `idauto`, `atelier_network`, `fixpert`	une seule base, 41 tables dans `public`
`docs/ARCHITECTURE.md` §1	Mythos OS fournit l'authentification, la facturation…	l'authentification est locale (Better Auth) ; il n'y a pas de facturation
`config/idauto.example.json`, champ `database.target_dbms_status`	« sélectionné — pas encore installé ni déployé »	PostgreSQL est déployé : ce champ est périmé. Les formats de plaque de ce fichier sont toujours lus.

Ces documents sont conservés sans modification. Le présent dossier décrit l'implémentation telle qu'elle est.

3. Diagramme de cas d'utilisation

Le diagramme ne montre que les fonctions **implémentées**. Les fonctions partielles, prévues ou abandonnées sont listées à la fin du chapitre et n'y figurent pas. Les sources sont `diagrams/use-case.mmd` (Mermaid) et `diagrams/use-case.puml` (PlantUML, notation UML standard).

3.1 Acteurs

Acteur	Nature	Comment le système le reconnaît	Source
Visiteur	humain, anonyme	aucun identifiant	`api.js` (`/public/*`, `/login`, pages publiques)
Technicien	humain	session Better Auth, `role = 'technician'`, `org_id` renseigné	`auth/principal.js`, `TECHNICIAN_SCOPES`
Gestionnaire	humain	session Better Auth, `role = 'manager'`, `org_id` renseigné	`auth/principal.js`, `MANAGER_SCOPES`
Administrateur	humain	session Better Auth, `role = 'admin'` (sans organisation)	`auth/principal.js` (`scopes: ['*']`)
Opérateur serveur	humain, sur le terminal du serveur	lance `node ops/auth-users.js` avec un accès à la base	`ops/auth-users.js`
Client de service	système externe	en-tête `Authorization: Bearer` avec un jeton listé dans `IDAUTO_ADMIN_IDENTITIES`	`reference/identity.js`

Deux éléments ne sont pas représentés comme acteurs :

- **Le propriétaire du véhicule.** Un atelier enregistre ses coordonnées, mais il ne se connecte jamais.
- **La session navigateur du propriétaire** (`/session`). C'est une aide de la console, installée par un administrateur et limitée à deux routes de lecture : c'est un mécanisme d'accès, pas un rôle.

3.2 Vue d'ensemble

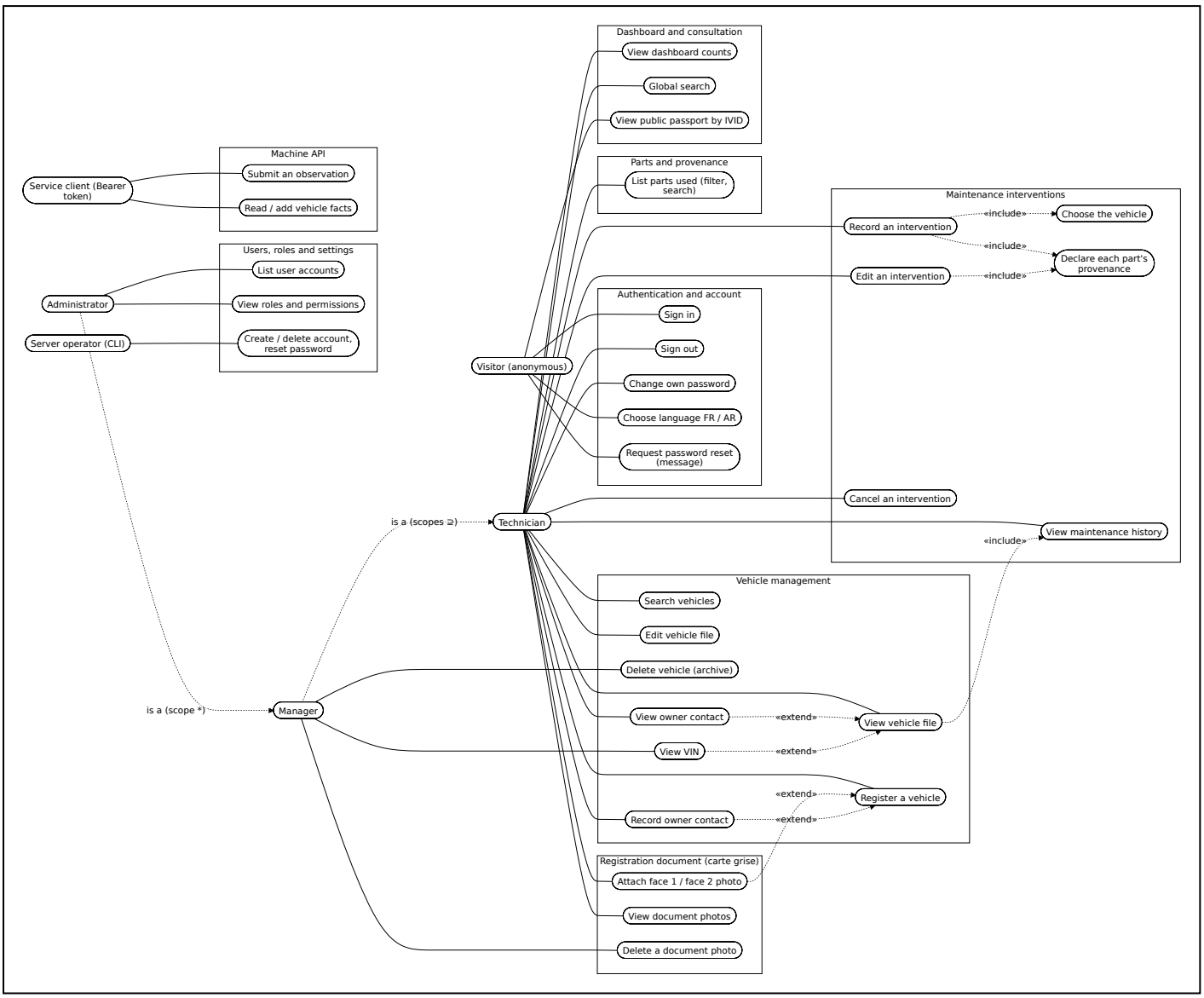


Figure 1 — Cas d'utilisation implémentés — vue d'ensemble

Source : `diagrams/use-case.mmd (+ use-case.puml)`

Chaque acteur n'est relié qu'aux cas d'utilisation que *son propre rôle* ajoute. Les flèches « is a » en pointillés sont des généralisations d'acteurs :

- le gestionnaire peut faire tout ce que fait le technicien ;
- l'administrateur peut faire tout ce que fait le gestionnaire.

Les relations «include» et «extend» sont tracées en pointillés. Mermaid n'a pas de notation native pour les cas d'utilisation, qui sont donc dessinés comme des ovales.

3.3 Vues détaillées

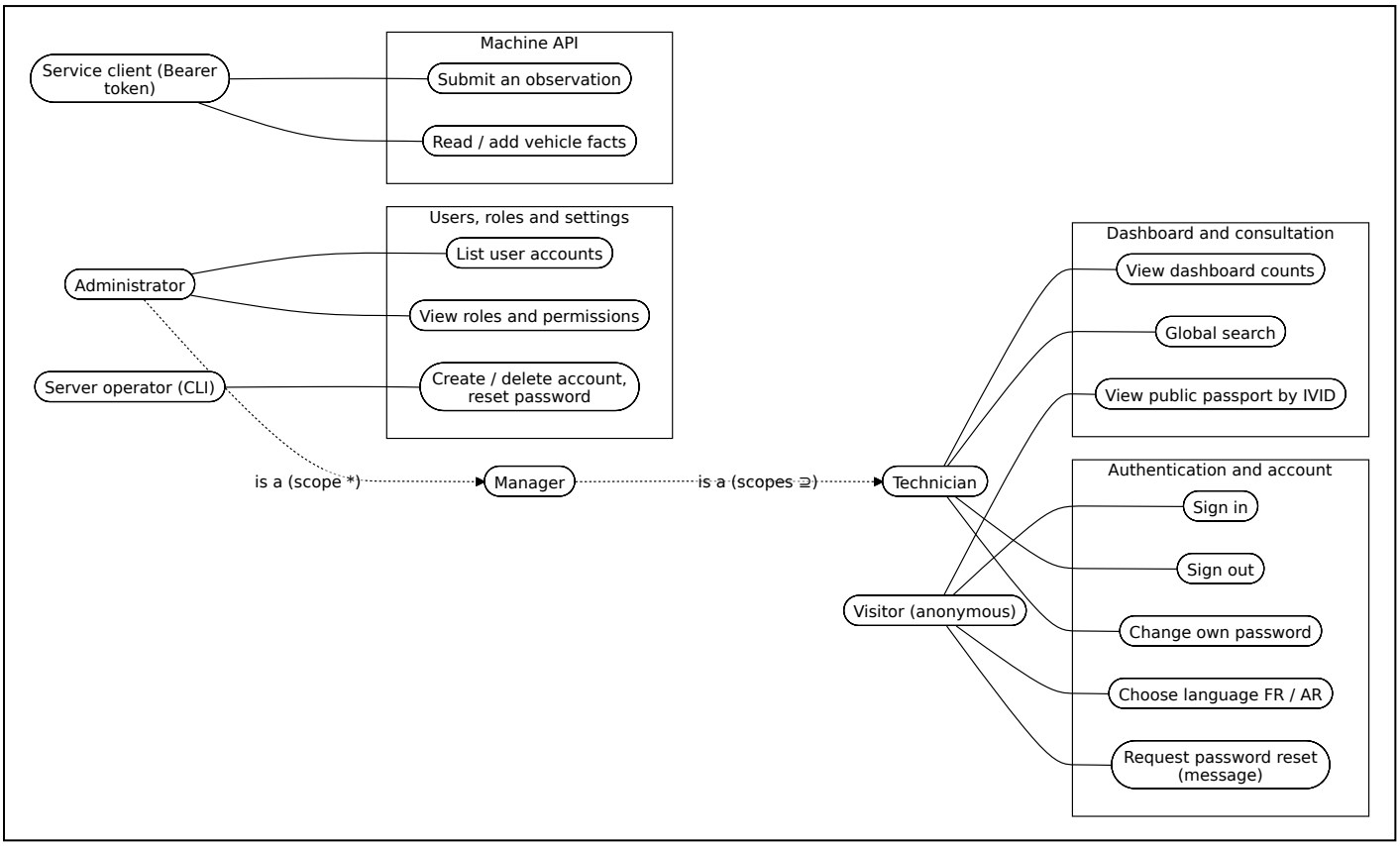


Figure 2 — Cas d'utilisation — authentification, consultation, administration et API machine

Source : `pdf/views/use-case-1.mmd — vue détaillée de diagrams/use-case.mmd`

Paquetages « Authentication and account », « Dashboard and consultation », « Users, roles and settings » et « Machine API », avec leurs acteurs.

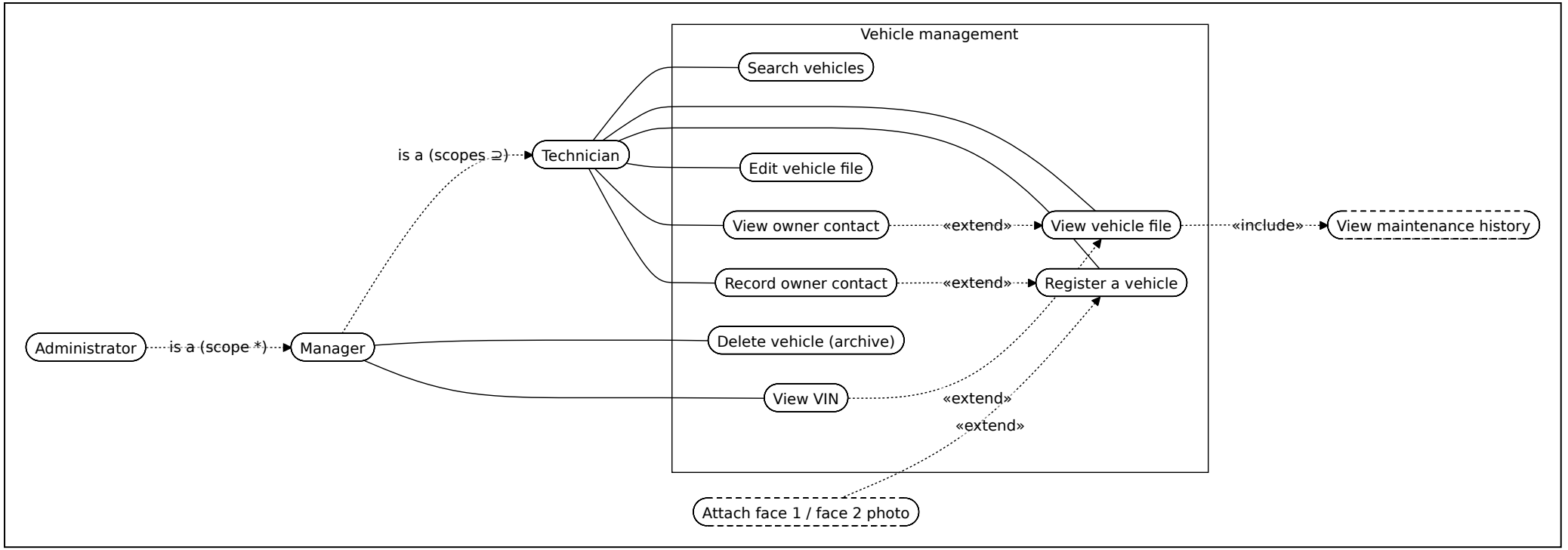


Figure 3 — Cas d'utilisation — gestion des véhicules

Source : `pdf/views/use-case-2.mmd — vue détaillée de diagrams/use-case.mmd`

Paquetage « Vehicle management ». Les cas en pointillés sont détaillés dans d'autres vues : « View maintenance history », inclus par « View vehicle file », et « Attach face 1 / face 2 photo », qui étend « Register a vehicle ».

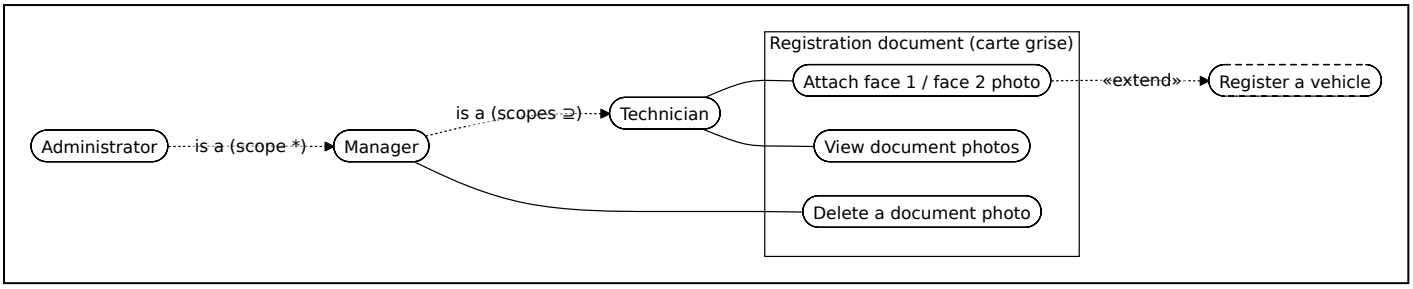


Figure 4 — Cas d'utilisation — carte grise

Source : `pdf/views/use-case-3.mmd — vue détaillée de diagrams/use-case.mmd`

Paquetage « Registration document ». Le cas en pointillés « Register a vehicle » appartient à la vue précédente.

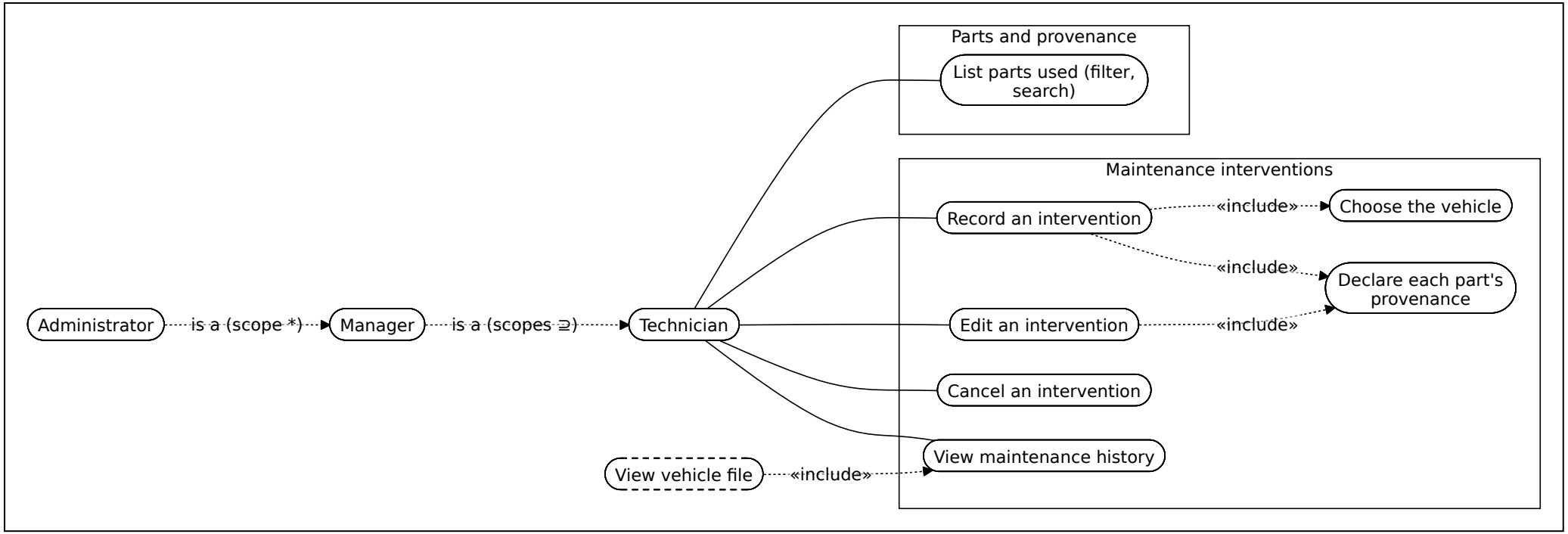


Figure 5 — Cas d'utilisation — interventions d'entretien et pièces

Source : `pdf/views/use-case-4.mmd — vue détaillée de diagrams/use-case.mmd`

Paquetages « Maintenance interventions » et « Parts and provenance ». Le cas en pointillés « View vehicle file » appartient à la vue « gestion des véhicules ».

3.4 Traçabilité : cas d'utilisation → route → droit

Le droit exigé vient de `ALL_ROUTE_SCOPES` (fichiers `api.js`, `workspace-routes.js`, `v14-routes.js` et `v12-routes.js`). Une route qui n'y figure pas est réservée à l'administrateur.

CU	Cas d'utilisation	Route(s)	Droit	Rôle minimal
UC1	Se connecter	`POST /api/auth/sign-in/email` (Better Auth, 5 par minute et par client)	aucun	Visiteur
UC2	Se déconnecter	`POST /api/auth/sign-out`	session	Technicien
UC3	Changer son mot de passe	`POST /api/auth/change-password` (les autres sessions sont révoquées)	session	Technicien
UC4	Choisir la langue	cookie `ida_lang`, lu par `i18n-serve.js`	aucun	Visiteur
UC5	Demander une réinitialisation du mot de passe	bouton « Mot de passe oublié » → message seulement (`login-ui.js`) ; ni route, ni courriel	aucun	Visiteur
UC10	Rechercher des véhicules	`GET /api/workspace/vehicles`	`vehicle:read`	Technicien
UC11	Consulter la fiche d'un véhicule	`GET /api/vehicles/:ref/dossier`	`vehicle:read`	Technicien
UC12	Enregistrer un véhicule	`POST /api/vehicles/register`	`vehicle:write`	Technicien
UC13	Modifier la fiche d'un véhicule	`PUT /api/vehicles/:ref/details`	`vehicle:write`	Technicien
UC14	Supprimer un véhicule (archivage)	`DELETE /api/vehicles/:ref`	`vehicle:delete`	Gestionnaire
UC15	Consulter le contact du propriétaire	`GET /api/vehicles/:ref/owner` (chaque lecture est auditée)	`owner:read`	Technicien
UC16	Enregistrer le contact du propriétaire	`PUT` et `DELETE /api/vehicles/:ref/owner`	`owner:write`	Technicien
UC17	Voir le VIN	renvoyé dans la fiche seulement avec `vin:search` ; chaque divulgation est auditée (`recordVinDisclosureAudit`)	`vin:search`	Gestionnaire
UC20	Joindre la photo du recto ou du verso	`PUT /api/vehicles/:ref/registration-document/:face` (et `/thumbnail`)	`document:write`	Technicien
UC21	Consulter les photos du document	`GET …/registration-document`, `GET …/:face/image`, `GET /api/workspace/documents`	`document:read`	Technicien
UC22	Supprimer une photo du document	`DELETE /api/vehicles/:ref/registration-document/:face`	`document:delete`	Gestionnaire
UC30–32	Saisir une intervention (véhicule, pièces, provenance)	`POST /api/workspace/interventions`	`workshop:write`	Technicien
UC33	Modifier une intervention	`PUT /api/workspace/interventions/:id`	`workshop:write`	Technicien
UC34	Annuler une intervention	`DELETE /api/workspace/interventions/:id` (statut → `cancelled`)	`workshop:write`	Technicien
UC35	Consulter l'historique d'entretien	inclus dans la fiche ; `GET /api/workspace/interventions`	`vehicle:read`, `workshop:read`	Technicien
UC40	Lister les pièces utilisées	`GET /api/workspace/parts-used`	`workshop:read`	Technicien
UC50	Consulter les compteurs du tableau de bord	`GET /api/workspace/overview`, `GET /api/notifications`	`vehicle:read`	Technicien

CU	Cas d'utilisation	Route(s)	Droit	Rôle minimal
UC51	Recherche globale	`GET /api/search`	`vehicle:read`	Technicien
UC52	Consulter le passeport public	`GET /public/passport/:ivid` (débit limité ; la recherche par plaque est fermée)	aucun	Visiteur
UC60	Lister les comptes utilisateurs	`GET /api/admin/accounts`	administrateur seulement	Administrateur
UC61	Consulter les rôles et les droits	`GET /api/admin/roles`	administrateur seulement (absente de la table des droits)	Administrateur
UC62	Créer ou supprimer un compte, réinitialiser un mot de passe	`ops/auth-users.js create`, `delete`, `set-password`, `set-role`, `revoke-sessions`	accès au terminal	Opérateur serveur
UC70	Soumettre une observation	`POST /api/observations`, `POST /api/ingest/observations`	`observation:write`	Client de service
UC71	Lire ou ajouter des faits sur un véhicule	`GET` et `POST /api/vehicles/:ref/facts`	`fact:read` / `fact:write`	Client de service

Cloisonnement par organisation. Chaque principal d'organisation ne voit que les lignes de sa propre organisation, et ce contrôle est refait dans les requêtes SQL. Deux cas sont particuliers :

- les contacts propriétaires et les notes sont privés à l'atelier qui les a saisis ;
- un véhicule est partagé entre organisations, mais son historique d'interventions est filtré par organisation.

3.5 Fonctions absentes du diagramme

Fonction	Statut	Raison
Lecture automatique de la carte grise (`…/registration-document/ocr`, `…/confirm`)	Abandonné	répond 410 sauf si `IDAUTO_CG_OCR_ENABLED=1`
Pages de la console d'exploitation (`/admin/*` : file de revue, fusion et séparation, qualité, sources)	Implémenté, hors du menu simplifié	réservées à la console. Leurs routes sont réservées à l'administrateur (`POST /api/review/*`, `POST /api/vehicles/:ref/merge`, `/split`). Elles doubleraient la taille du diagramme ; elles sont décrites dans l'architecture logique.
Ancienne page atelier `/atelier` (photo de plaque, OCR dans le navigateur, visites V12)	Implémenté, hors du menu simplifié	remplacée par les cas d'utilisation des interventions
Commandes et stock d'atelier	Partiel	tables et fonctions de dépôt seulement
Consultation du catalogue TecDoc	Partiel, non configuré	aucune configuration sur le serveur de production
Caméras, mouvements, consentements, vérifications	Prévu	tables seulement, aucun chemin de code
Réinitialisation du mot de passe en libre-service par courriel	Non implémenté	pas d'intégration de courriel

4. Diagramme de classes

4.1 Nature des classes

IDauto est écrit sous forme de modules CommonJS qui exportent des fonctions. Le code **ne définit aucune classe JavaScript, aucun modèle d'ORM et aucun type TypeScript** pour le domaine : les lignes circulent sous forme d'objets simples construits à partir des résultats SQL. Le chapitre présente donc deux vues distinctes, sans les confondre avec les tables :

- **Vue A — entités du domaine.** Ce sont les classes conceptuelles que l'application lit et écrit. Chacune repose sur exactement une table. Les noms et les types des attributs sont copiés de `database/schema.sql` et des migrations. Seuls les attributs utilisés par le code sont montrés ; les listes complètes de colonnes sont dans `06-data-model.md`.
- **Vue B — modules de service.** Ce sont les unités de code et les fonctions qu'elles exportent réellement (`Object.keys(require(module))`).

4.2 Vue A — entités du domaine

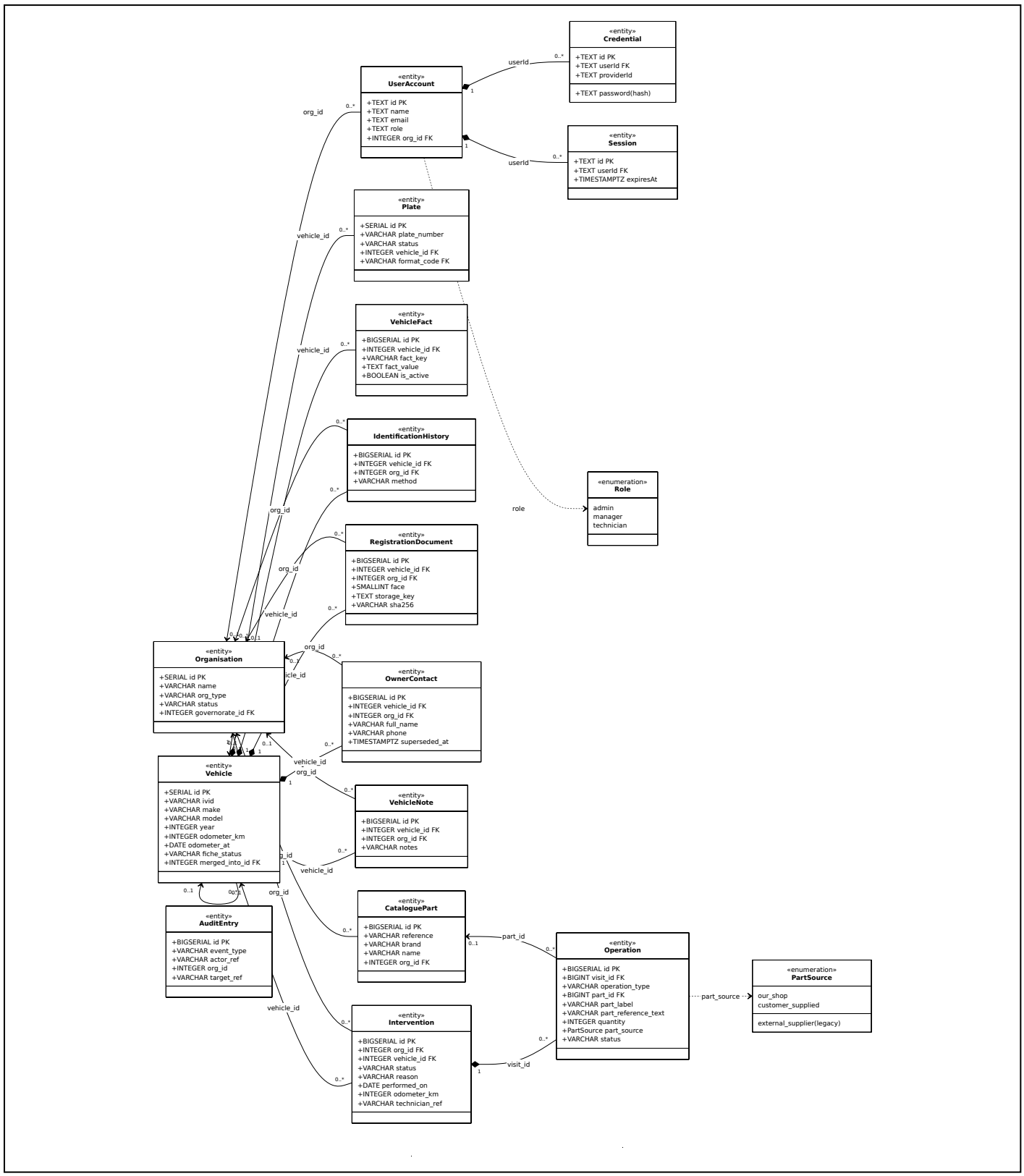


Figure 6 — Entités du domaine utilisées par l'application — vue d'ensemble

Source : `diagrams/class-diagram.mmd (+ class-diagram.puml)`

La notation se lit ainsi :

- **Composition** (losange plein) : la ligne enfant porte une clé étrangère **NOT NULL** vers le parent et n'a pas de sens sans lui.
- **Association** (flèche) : la clé étrangère est facultative. La multiplicité côté parent vaut `0..1` si la clé peut être nulle, `1` si elle est NOT NULL.
- **Dépendance vers une «enumeration»** : colonne contrainte par un CHECK en base.

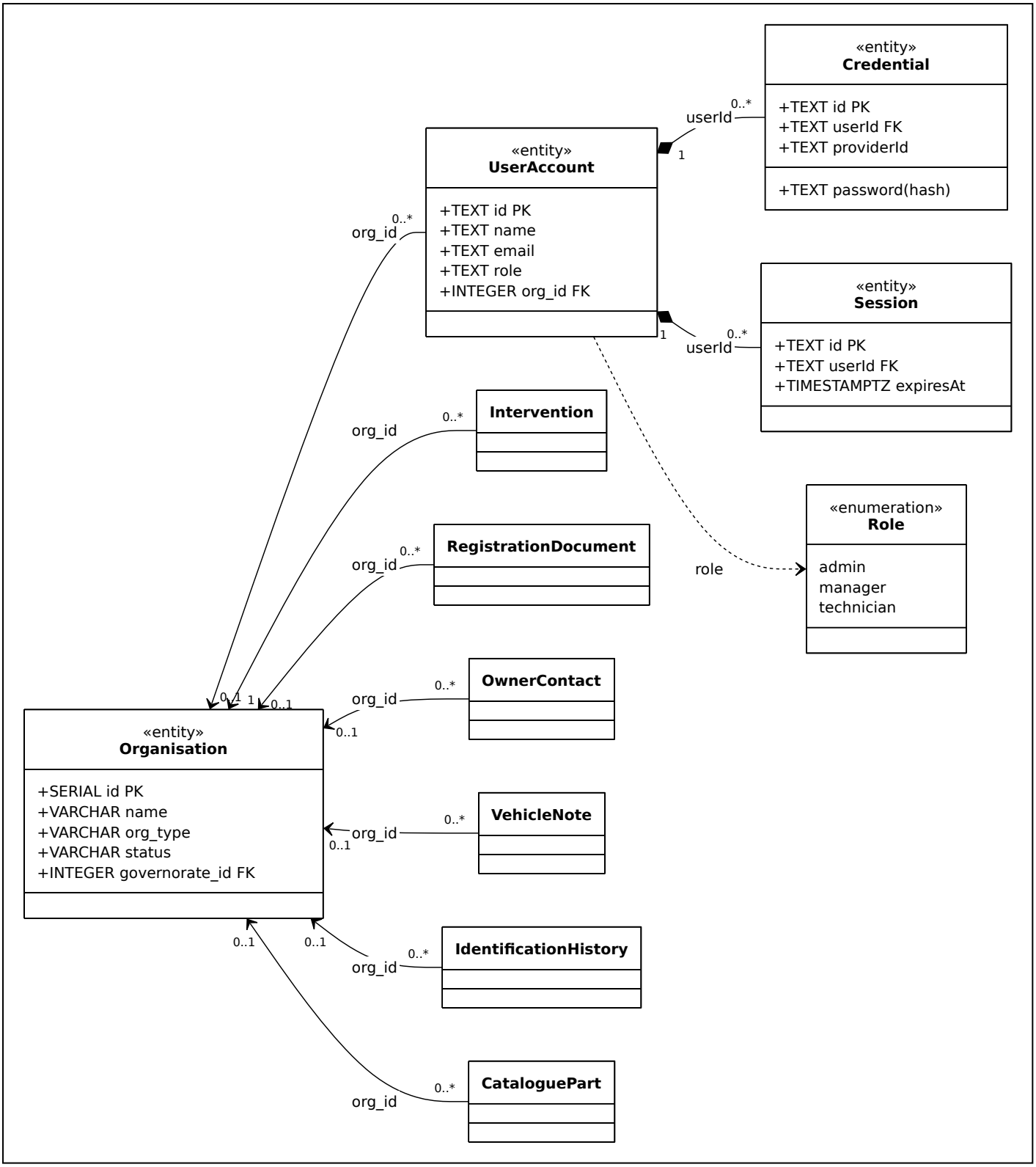


Figure 7 — Entités — organisations, comptes et sessions

Source : `pdf/views/class-diagram-1.mmd — vue détaillée de diagrams/class-diagram.mmd`

`Role` est contraint par le CHECK `chk_auth_user_role` (migration `ida-v13-auth-session.sql`). Better Auth gère les colonnes des trois tables d'authentification ; IDauto y ajoute `role` et `org_id`.

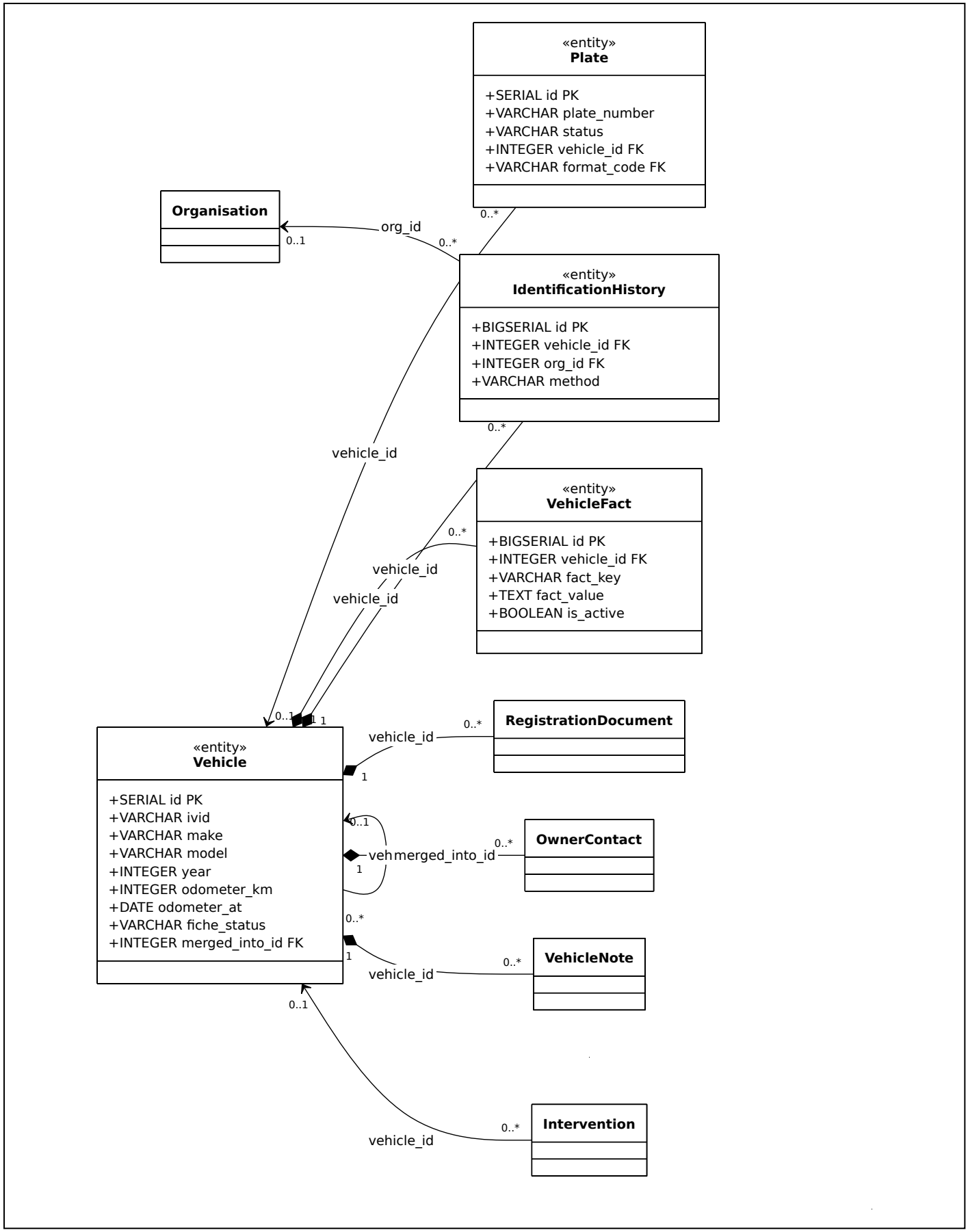


Figure 8 — Entités — véhicule, plaques, faits et historique d'identification

Source : `pdf/views/class-diagram-2.mmd — vue détaillée de diagrams/class-diagram.mmd`

Le VIN est un fait (`VehicleFact`). Une plaque peut ne pas être rattachée à un véhicule (`0..1`). La relation de `Vehicle` vers lui-même (`merged_into_id`) représente la fusion de deux fiches.

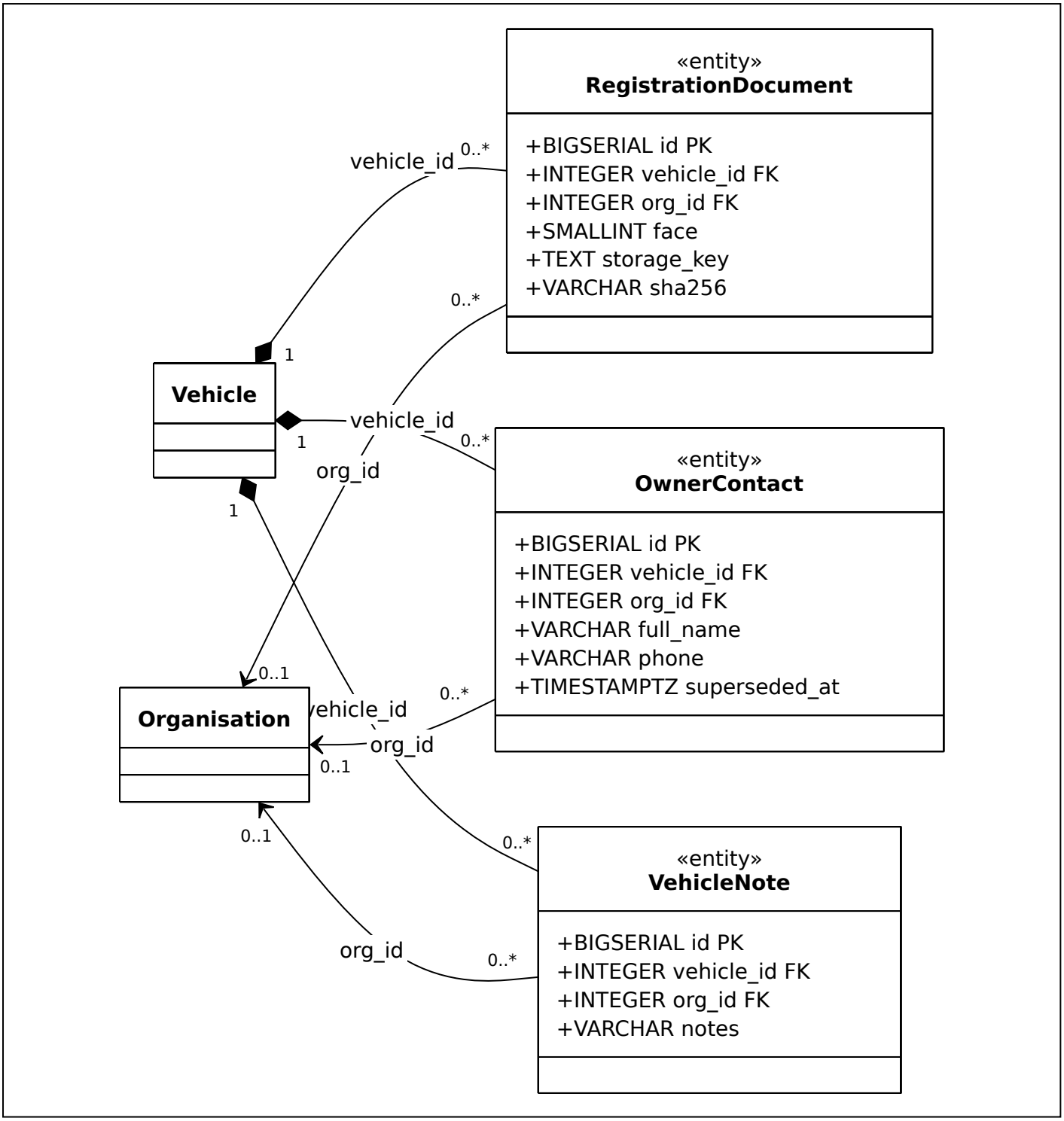


Figure 9 — Entités — carte grise, contact propriétaire et note d'atelier

Source : `pdf/views/class-diagram-3.mmd — vue détaillée de diagrams/class-diagram.mmd`

Ces trois entités sont des compositions du véhicule. Elles sont rattachées facultativement à l'organisation qui les a saisies.

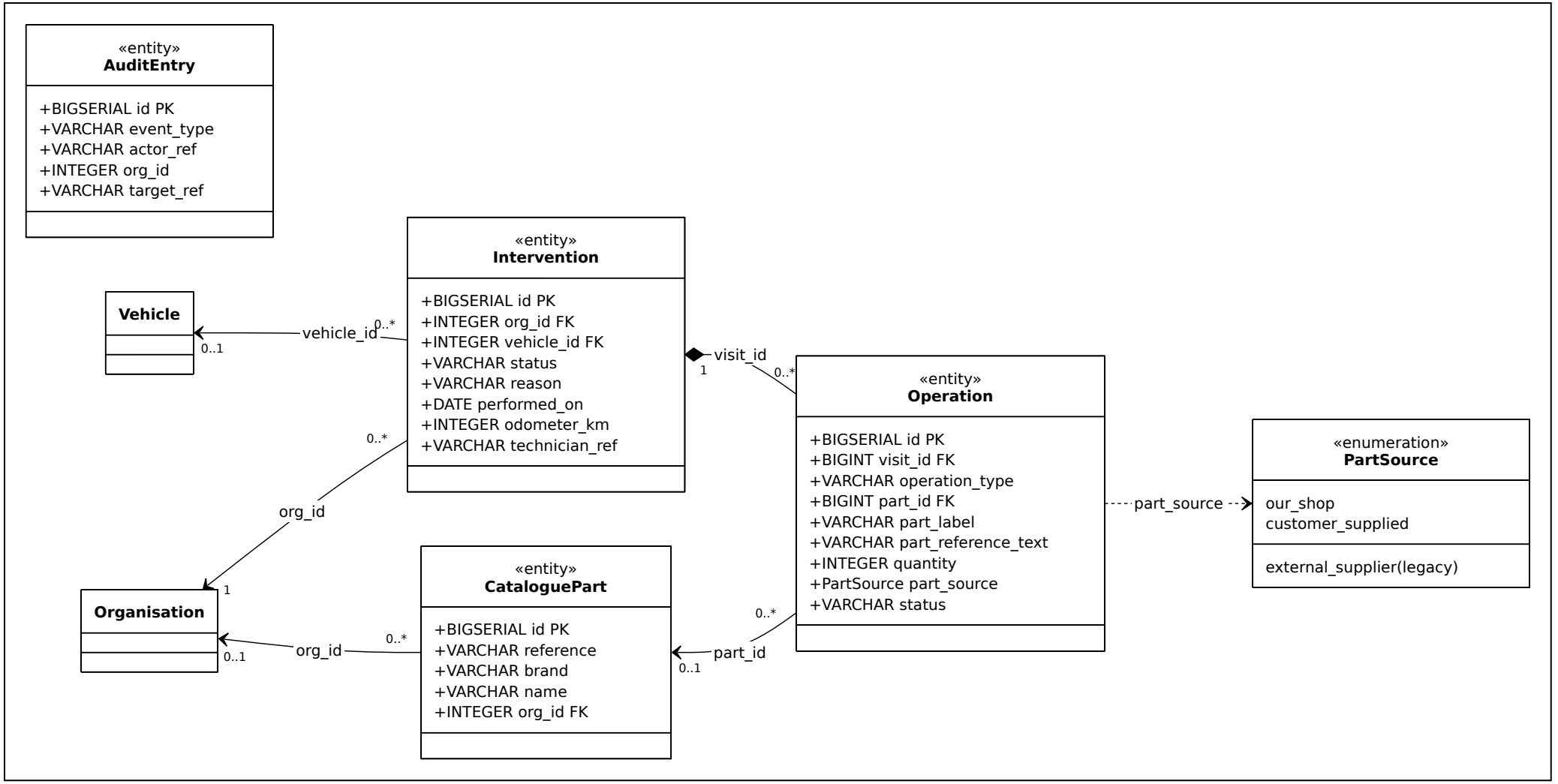


Figure 10 — Entités — interventions, opérations, pièces et audit

Source : `pdf/views/class-diagram-4.mmd — vue détaillée de diagrams/class-diagram.mmd`

Une intervention appartient obligatoirement à une organisation et se compose d'opérations. La provenance `PartSource` a deux valeurs dans l'application ; `external_supplier` n'est plus accepté que pour les lignes anciennes. `AuditEntry` n'a volontairement aucune clé étrangère.

4.3 Règles métier invisibles sur le diagramme

- **Suppression d'un véhicule = archivage.** `fiche_status = 'archived'`, les plaques sont annulées et le fait VIN est désactivé. Rien n'est effacé physiquement (`vehicle-details.js remove()`).
- **Une seule plaque active par numéro.** C'est un index unique partiel, `uidx_idauto_plates_number_active`.
- **Un seul contact propriétaire courant par véhicule et par atelier.** L'index `uq_owner_contact_current` porte sur les lignes où `superseded_at IS NULL`. Un nouveau contact remplace l'ancien, qui est conservé.
- **Une note par véhicule et par atelier** (`uq_vehicle_notes_org`).
- **Annuler une intervention** fixe `status = 'cancelled'` ; ce n'est jamais un effacement.
- **Chaque écriture est auditée.** `AuditEntry` n'a volontairement aucune clé étrangère : il enregistre des références (`actor_ref`, `target_ref`, `org_id`) sous forme de texte et de nombres, pour survivre aux suppressions. Chaque ligne est écrite dans la même transaction que la modification qu'elle décrit (`writes.withAudit`).

4.4 Correspondance entité → table → code

Entité	Table	Code principal qui la lit ou l'écrit
Organisation	`idauto_organizations`	`intervention-service.orgOf`, `admin-repository`, `principal.js` (par `org_id`)
UserAccount	`idauto_auth_user`	Better Auth (`auth/auth.js`), `ops/auth-users.js`, `workspace-repository.listTechnicians`
Credential	`idauto_auth_account`	Better Auth (empreinte du mot de passe, scrypt)
Session	`idauto_auth_session`	Better Auth
Vehicle	`idauto_vehicles`	`vehicle-repository`, `registration-service`, `vehicle-details`, `writes.createVehicle`, `mergeVehicle`, `splitVehicle`
Plate	`idauto_plates`	`vehicle-repository.attachPlate`, `findByPlate`, `writes.createPlate`
VehicleFact	`idauto_vehicle_facts`	`vehicle-repository.vinOf` (le VIN est un fait), `writes.createFact`, `reviewFact`
IdentificationHistory	`idauto_vehicle_identification_history`	`vehicle-repository.insertHistory`, `history`
RegistrationDocument	`idauto_vehicle_documents`	`documents/document-repository`, `document-service.store`, `list`, `remove`
OwnerContact	`idauto_vehicle_owner_contacts`	`owner-contacts.put`, `current`, `remove`, `search`
VehicleNote	`idauto_vehicle_notes`	`vehicle-details.upsertNotesWith`, `notesOf`
Intervention	`idauto_workshop_visits`	`intervention-service.create`, `update`, `cancel` ; `workshop-repository.createVisit`
Operation	`idauto_workshop_operations`	`intervention-service.insertParts`, `workshop-repository.addOperation`
CataloguePart	`idauto_parts`	`parts/local-parts-catalog.js`, `workshop-repository`
AuditEntry	`idauto_audit_log`	`writes.withAudit` et chaque fonction `record*Audit`

Contrôle automatique. Un script a confronté le diagramme au schéma analysé :

- chaque attribut (nom et type SQL de base) ;
- chaque relation, avec une clé étrangère réellement appliquée ;
- chaque multiplicité côté parent, avec la nullabilité de cette clé.

Résultat : **102 éléments vérifiés, 0 écart**.

Hors de la vue A, volontairement :

- les 9 tables qu'aucun code n'utilise (prévues) ;
- les tables d'observations, de preuves et de revue, utilisées par l'API machine et la console ;
- les tables de référence (gouvernorats, formats de plaque).

Toutes figurent au chapitre « Modèle de données ».

4.5 Vue B — modules de service

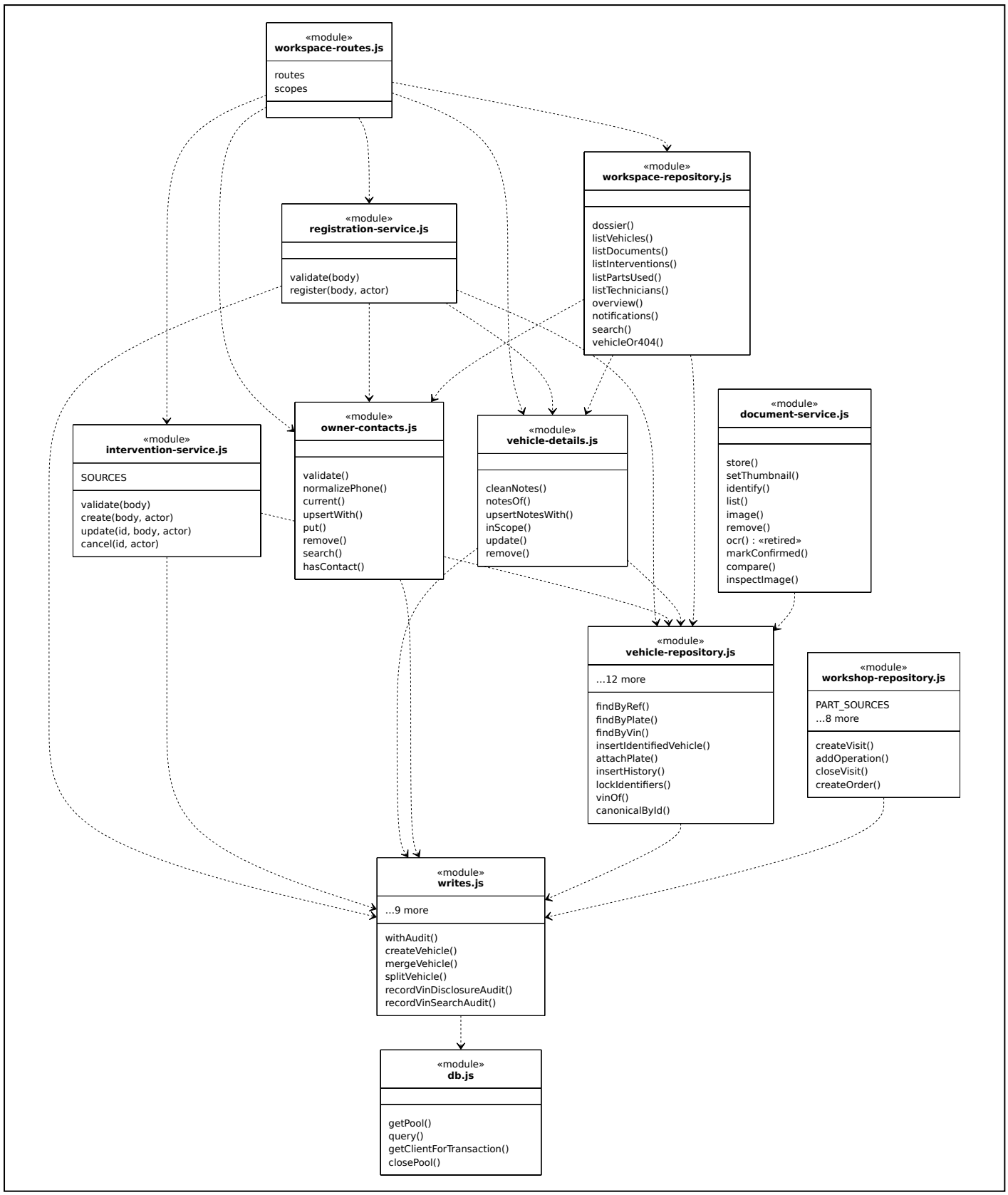


Figure 11 — Modules de service et fonctions exportées — vue d'ensemble

Source : `diagrams/class-modules.mmd (copie de 03-class-diagram.md)`

- Chaque boîte est un fichier, pas une classe instanciable. Ses membres sont les noms que le fichier exporte.
- Les flèches en pointillés sont des dépendances `require()`, toutes lues dans le code source.
- Chaque module qui accède à la base requiert aussi `db.js` directement. Seule la flèche `writes → db` est tracée, par souci de lisibilité.

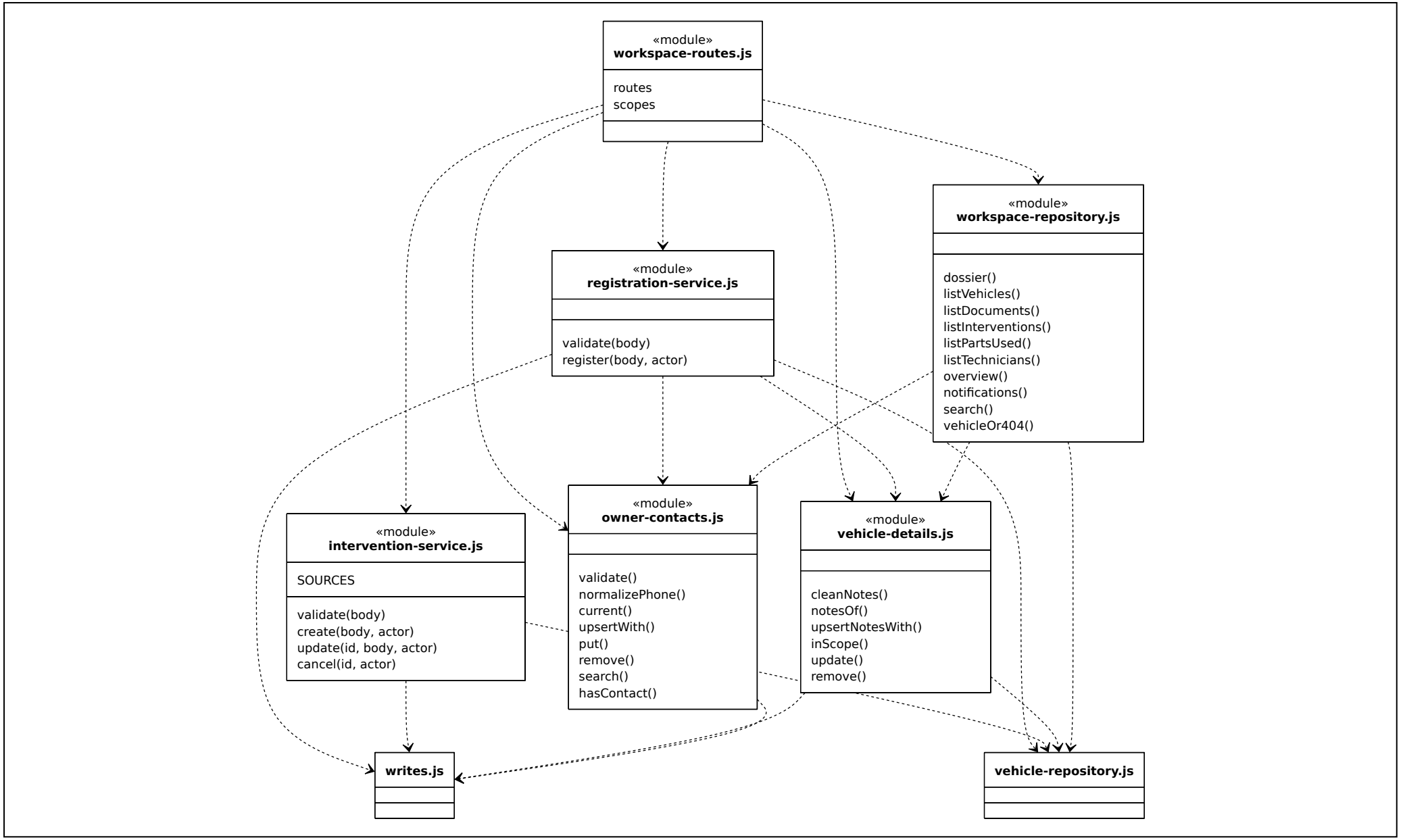


Figure 12 — Modules — routes de l'espace de travail et services métier

Source : `pdf/views/class-modules-1.mmd — vue détaillée de diagrams/class-modules.mmd`

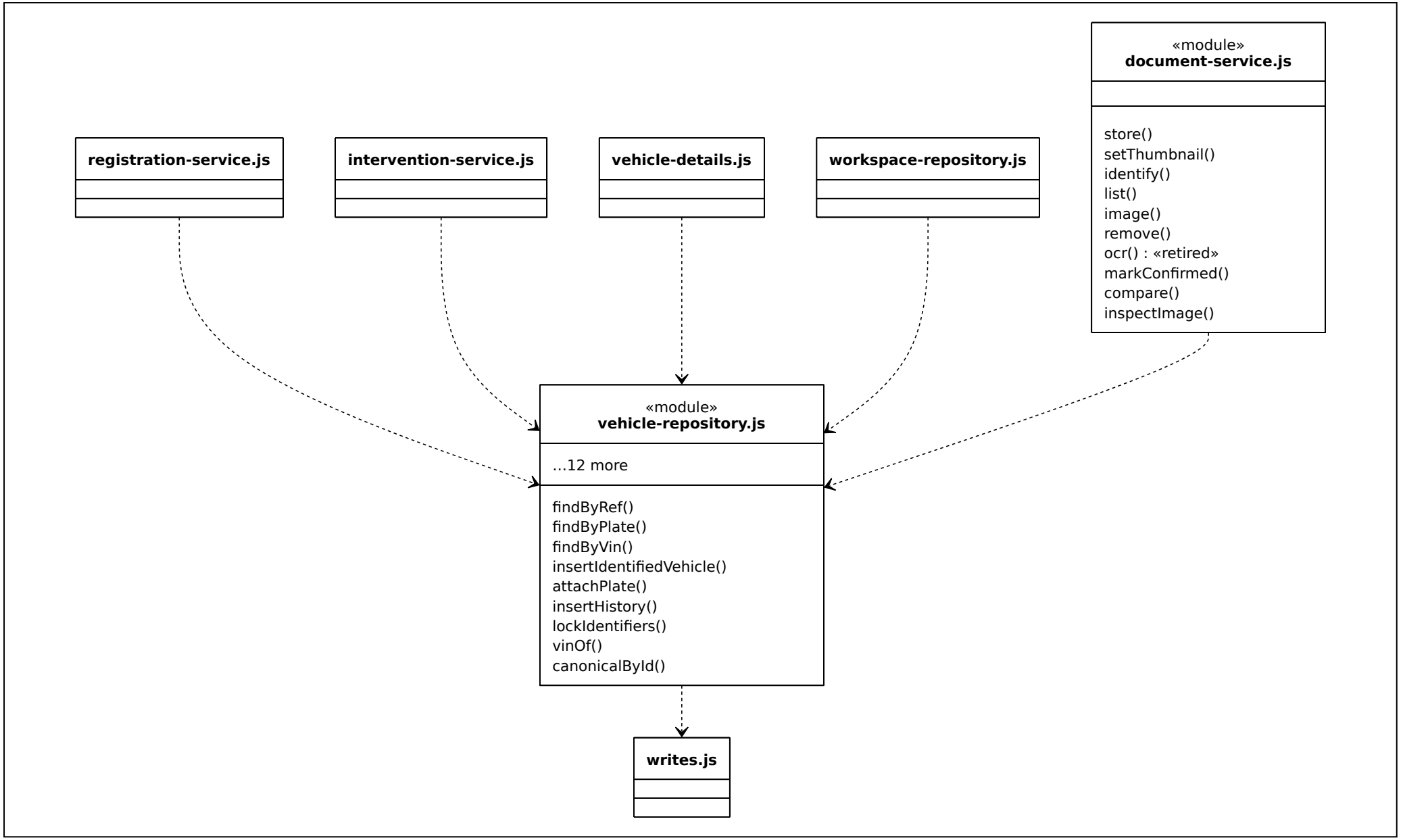


Figure 13 — Modules — service des documents et dépôt des véhicules

Source : `pdf/views/class-modules-2.mmd — vue détaillée de diagrams/class-modules.mmd`

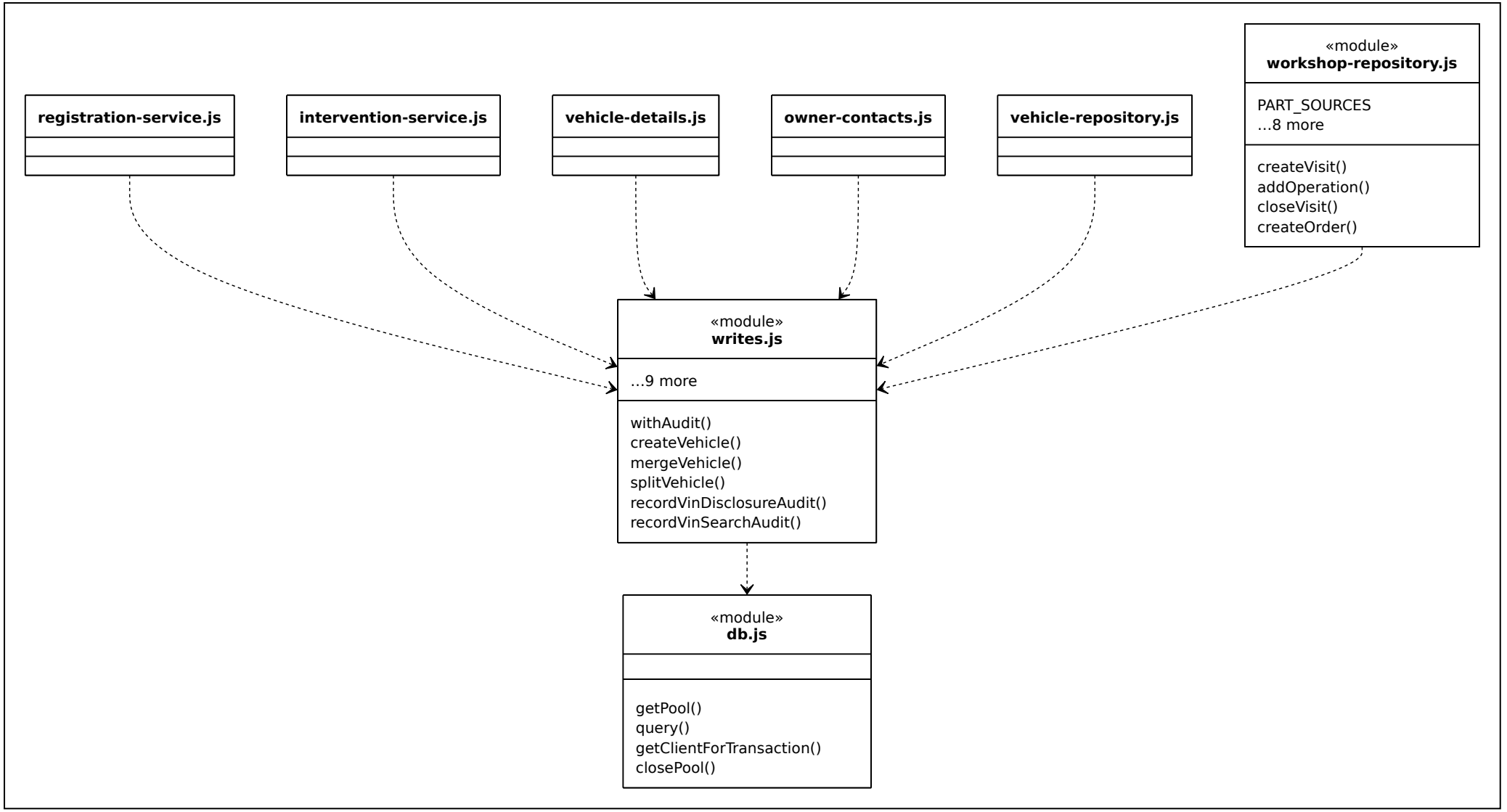


Figure 14 — Modules — dépôt de l'atelier, écritures auditées et accès à la base

Source : `pdf/views/class-modules-3.mmd — vue détaillée de diagrams/class-modules.mmd`

- `document-service` enregistre les fichiers par `document-storage.js` et les métadonnées par `document-repository.js`.
- `intervention-service` écrit les tables des visites et des opérations avec son propre SQL, à l'intérieur de `writes.withAudit`. Il n'appelle pas `workshop-repository`, qui sert l'ancienne page atelier (V12).

5. Modèle de données

5.1 Principes

- **Source.** `database/schema.sql` et les migrations de `database/migrations/`, appliquées dans l'ordre de `docs/DATABASE.md`. Le modèle est **généré à partir du SQL** ; rien n'y est dessiné à la main.
- **Nombre de tables.** 41 tables, toutes dans le schéma `public` de la base `idauto_production`. La liste des tables en production est identique à celle du dépôt (41 = 41). Les schémas cibles décrits dans `docs/ARCHITECTURE.md` §2 ne sont pas implémentés.
- **Utilisation.** Une table marquée « schéma seulement » est créée par le schéma, mais aucun chemin de code de `reference/` ou d'`ops/` ne la lit ni ne l'écrit.
- **Clés étrangères.** Seules les contraintes appliquées par PostgreSQL (`REFERENCES`) sont tracées. Une relation décrite seulement par un commentaire est listée à part.
- **Multiplicités.** `||` marque une clé étrangère obligatoire (NOT NULL), `|o` une clé facultative.
- **Colonnes affichées.** Chaque diagramme ER montre les clés et les principales colonnes métier ; la liste complète des colonnes est dans `06-data-model.md`.

Répartition : **32 tables utilisées par le code**, **9 tables « schéma seulement »**.

5.2 Inventaire des tables

Identité, comptes et organisations

Table	Statut	Clé primaire	Clés étrangères appliquées	Définie dans
`idauto_auth_user`	utilisée par le code	`id`	`org_id` → `idauto_organizations`	`database/migrations/ida-v13-auth-session.sql`
`idauto_auth_account`	utilisée par le code	`id`	`userId` → `idauto_auth_user`	`database/migrations/ida-v13-auth-session.sql`
`idauto_auth_session`	utilisée par le code	`id`	`userId` → `idauto_auth_user`	`database/migrations/ida-v13-auth-session.sql`
`idauto_auth_verification`	utilisée par le code	`id`	—	`database/migrations/ida-v13-auth-session.sql`
`idauto_auth_rate_limit`	utilisée par le code	`id`	—	`database/migrations/ida-v13-auth-session.sql`
`idauto_organizations`	utilisée par le code	`id`	`governorate_id` → `idauto_governorates`	`database/schema.sql`
`idauto_user_roles`	**schéma seulement (prévue)**	`id`	`org_id` → `idauto_organizations`	`database/schema.sql`
`idauto_governorates`	utilisée par le code	`id`	—	`database/schema.sql`

Registre des véhicules, documents et données privées d'atelier

Table	Statut	Clé primaire	Clés étrangères appliquées	Définie dans
`idauto_vehicles`	utilisée par le code	`id`	`merged_into_id` → `idauto_vehicles`	`database/schema.sql`
`idauto_plates`	utilisée par le code	`id`	`format_code` → `idauto_plate_formats`, `governorate_id` → `idauto_governorates`, `vehicle_id` → `idauto_vehicles`	`database/schema.sql`
`idauto_plate_formats`	utilisée par le code	`id`	—	`database/schema.sql`

Table	Statut	Clé primaire	Clés étrangères appliquées	Définie dans
idauto_vehicle_facts	utilisée par le code	id	vehicle_id → idauto_vehicles, source_id → idauto_capture_sources, observation_id → idauto_observations	database/schema.sql
idauto_fact_evidence	utilisée par le code	id	fact_id → idauto_vehicle_facts	database/schema.sql
idauto_vehicle_identification_history	utilisée par le code	id	vehicle_id → idauto_vehicles, org_id → idauto_organizations	database/migrations/ida-v12-vehicle-identification.sql
idauto_vehicle_documents	utilisée par le code	id	vehicle_id → idauto_vehicles, org_id → idauto_organizations	database/migrations/ida-v14-registration-document.sql
idauto_vehicle_owner_contacts	utilisée par le code	id	vehicle_id → idauto_vehicles, org_id → idauto_organizations	database/migrations/ida-ui3-registration-maintenance.sql
idauto_vehicle_notes	utilisée par le code	id	vehicle_id → idauto_vehicles, org_id → idauto_organizations	database/migrations/ida-ui4-simplification.sql
idauto_verifications	**schéma seulement (prévue)**	id	vehicle_id → idauto_vehicles, org_id → idauto_organizations, user_role_id → idauto_user_roles	database/schema.sql
idauto_service_events	utilisée par le code	id	vehicle_id → idauto_vehicles, plate_id → idauto_plates, org_id → idauto_organizations, user_role_id → idauto_user_roles	database/schema.sql

Entretien en atelier, pièces et provenance

Table	Statut	Clé primaire	Clés étrangères appliquées	Définie dans
idauto_workshop_visits	utilisée par le code	id	org_id → idauto_organizations, vehicle_id → idauto_vehicles, customer_ref_id → idauto_workshop_customer_refs	database/migrations/ida-v12-vehicle-identification.sql
idauto_workshop_operations	utilisée par le code	id	visit_id → idauto_workshop_visits, part_id → idauto_parts	database/migrations/ida-v12-vehicle-identification.sql
idauto_workshop_customer_refs	utilisée par le code	id	org_id → idauto_organizations	database/migrations/ida-v12-vehicle-identification.sql
idauto_parts	utilisée par le code	id	org_id → idauto_organizations	database/migrations/ida-v12-vehicle-identification.sql
idauto_part_compatibility	utilisée par le code	id	part_id → idauto_parts, vehicle_id → idauto_vehicles	database/migrations/ida-v12-vehicle-identification.sql
idauto_org_stock	utilisée par le code	id	org_id → idauto_organizations, part_id → idauto_parts	database/migrations/ida-v12-vehicle-identification.sql
idauto_workshop_orders	utilisée par le code	id	org_id → idauto_organizations, visit_id → idauto_workshop_visits	database/migrations/ida-v12-vehicle-identification.sql
idauto_workshop_order_lines	utilisée par le code	id	order_id → idauto_workshop_orders, part_id → idauto_parts	database/migrations/ida-v12-vehicle-identification.sql

Capture, réception et revue

Table	Statut	Clé primaire	Clés étrangères appliquées	Définie dans
idauto_observations	utilisée par le code	id	vehicle_id → idauto_vehicles, plate_id → idauto_plates, capture_source_id → idauto_capture_sources, capture_session_id → idauto_capture_sessions, camera_source_id → idauto_camera_sources, contributor_id → idauto_contributors, org_id → idauto_organizations	database/schema.sql
idauto_observation_media	utilisée par le code	id	observation_id → idauto_observations, derived_from_media_id → idauto_observation_media	database/schema.sql
idauto_observation_locations	**schéma seulement (prévue)**	id	observation_id → idauto_observations, governorate_id → idauto_governorates	database/schema.sql
idauto_capture_sources	utilisée par le code	id	—	database/schema.sql
idauto_capture_sessions	**schéma seulement (prévue)**	id	contributor_id → idauto_contributors, capture_source_id → idauto_capture_sources	database/schema.sql
idauto_contributors	utilisée par le code	id	—	database/schema.sql
idauto_camera_sources	**schéma seulement (prévue)**	id	capture_source_id → idauto_capture_sources	database/schema.sql
idauto_vehicle_movements	**schéma seulement (prévue)**	id	observation_id → idauto_observations, vehicle_id → idauto_vehicles, plate_id → idauto_plates, camera_source_id → idauto_camera_sources	database/schema.sql
idauto_submissions	utilisée par le code	id	capture_source_id → idauto_capture_sources, observation_id → idauto_observations	database/migrations/ida-3a-ingestion-schema.sql
idauto_review_queue	**schéma seulement (prévue)**	id	vehicle_id → idauto_vehicles	database/schema.sql
idauto_document_scans	**schéma seulement (prévue)**	id	observation_id → idauto_observations, vehicle_id → idauto_vehicles, media_id → idauto_observation_media	database/schema.sql
idauto_consent_records	**schéma seulement (prévue)**	id	—	database/schema.sql

Éléments transverses : audit et limitation de débit

Table	Statut	Clé primaire	Clés étrangères appliquées	Définie dans
idauto_audit_log	utilisée par le code	id	—	database/schema.sql
idauto_rate_limit_counters	utilisée par le code	bucket_key, window_start	—	database/migrations/ida-3a-ingestion-schema.sql

5.3 Diagrammes entité-association

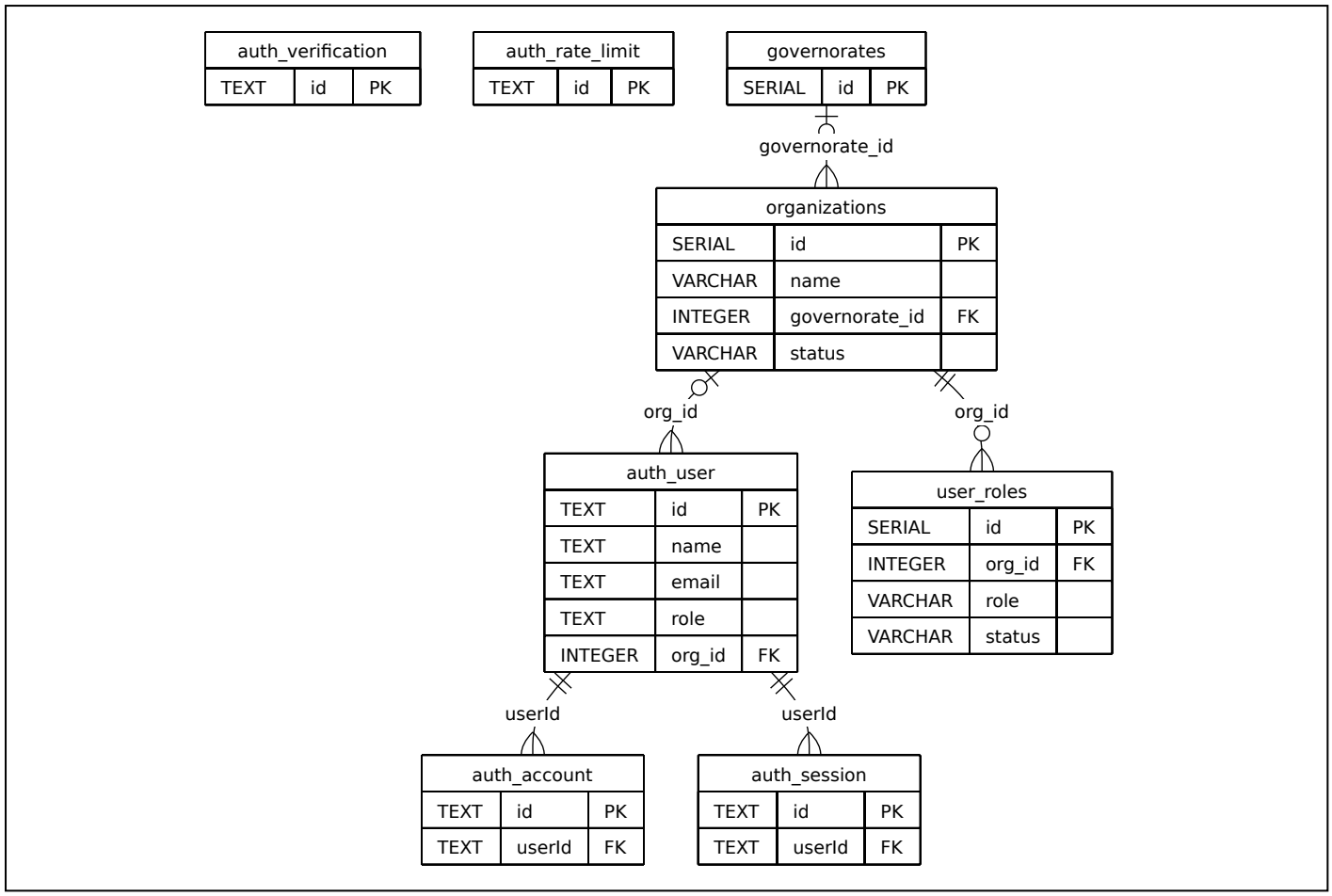


Figure 15 — Modèle de données — identité, comptes et organisations

Source : `diagrams/er-identity.mmd`

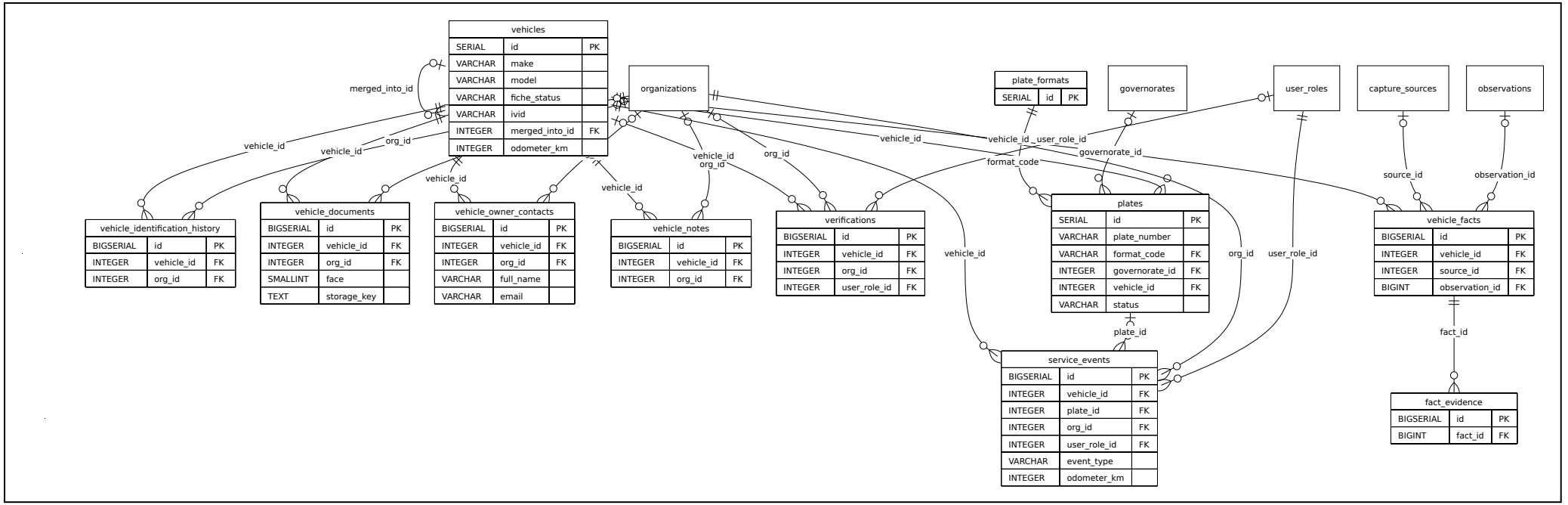


Figure 16 — Modèle de données — registre des véhicules, documents et données privées d'atelier (vue d'ensemble)

Source : `diagrams/er-vehicle.mmd`

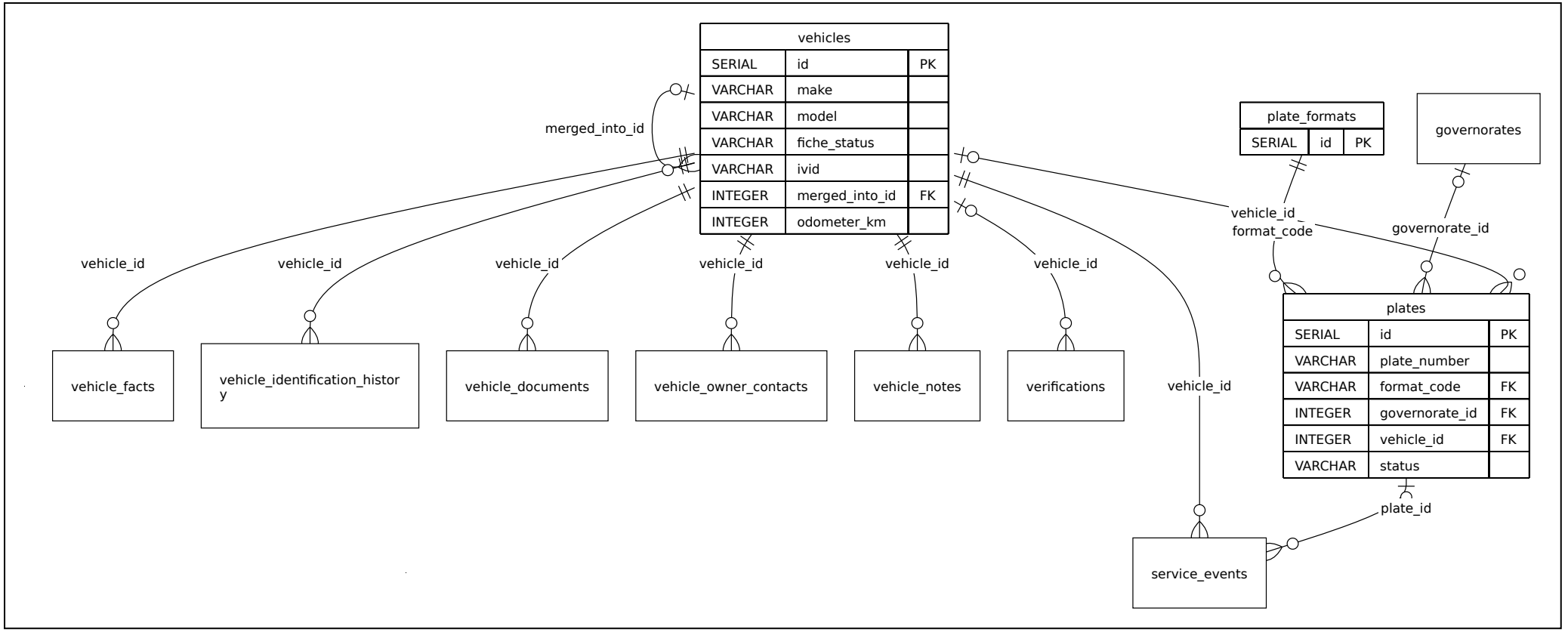


Figure 17 — Modèle de données — véhicules et plaques

Source : `pdf/views/er-vehicle-1.mmd — vue détaillée de diagrams/er-vehicle.mmd`

Une table dessinée sans colonnes (ici les gouvernorats) est détaillée dans un autre diagramme.

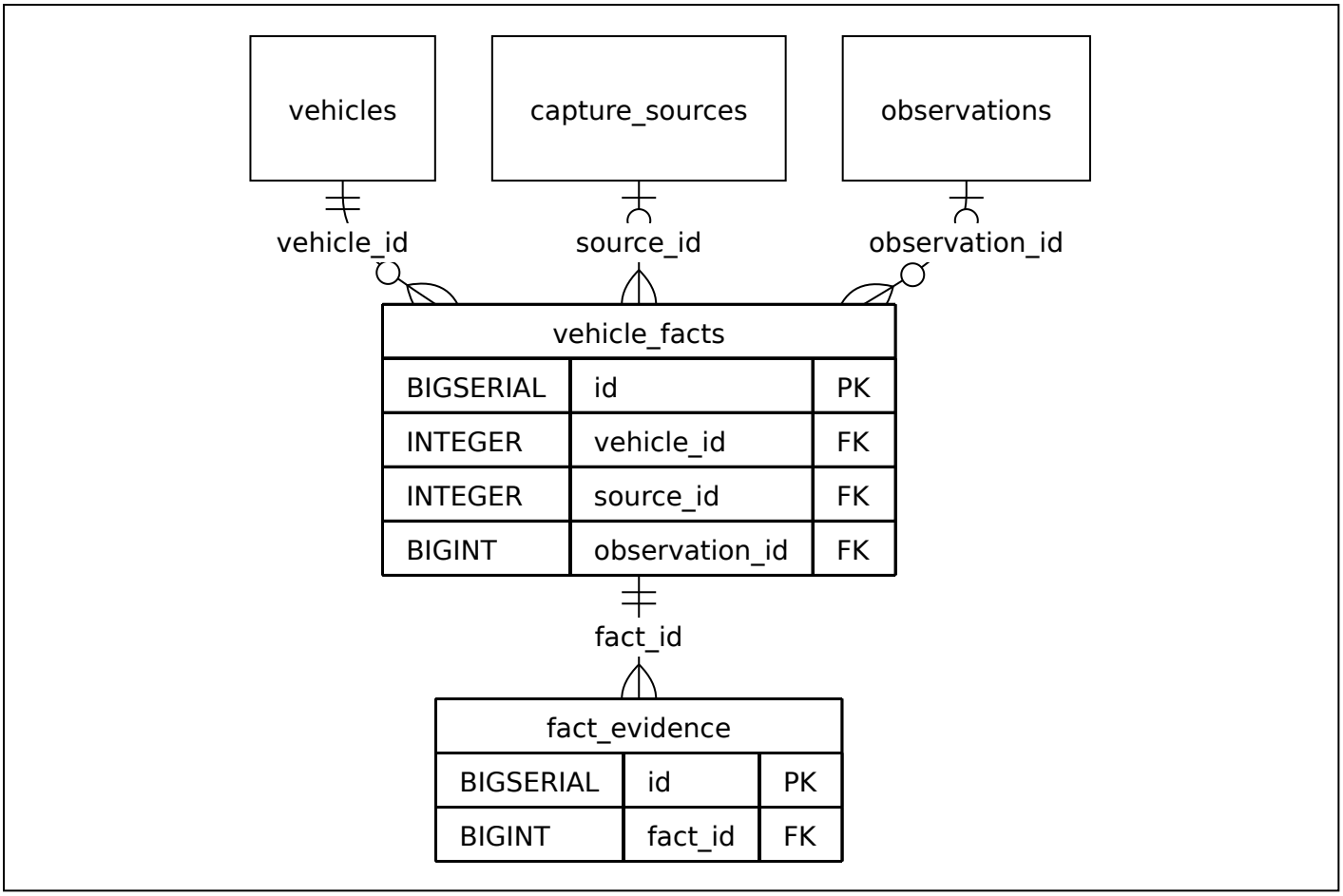


Figure 18 — Modèle de données — faits et preuves

Source : `pdf/views/er-vehicle-2.mmd — vue détaillée de diagrams/er-vehicle.mmd`

Les sources de capture et les observations sont détaillées dans le diagramme « capture, réception et revue ».

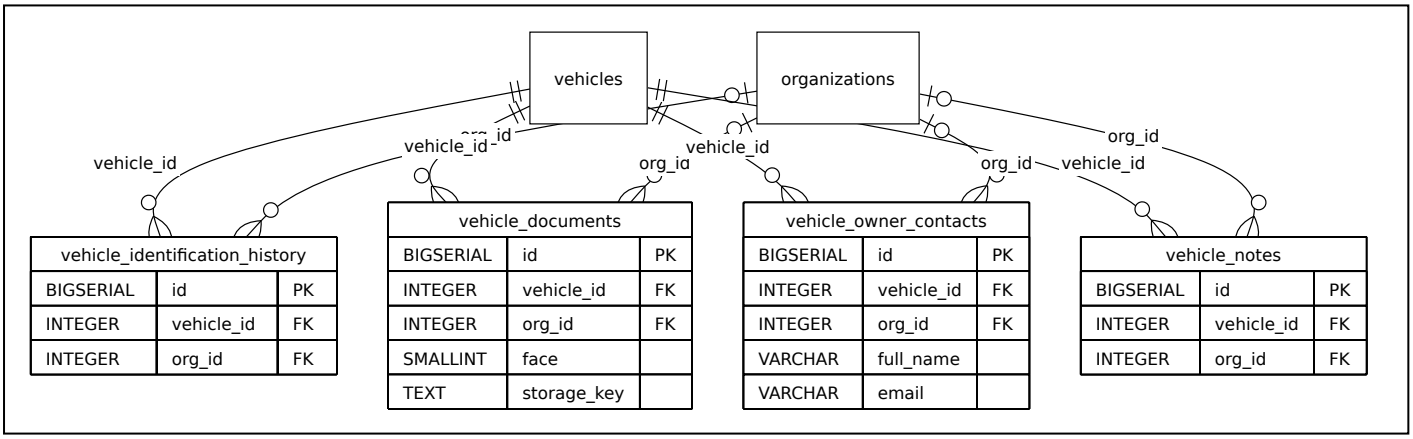


Figure 19 — Modèle de données — historique, carte grise, contacts et notes par organisation

Source : `pdf/views/er-vehicle-3.mmd — vue détaillée de diagrams/er-vehicle.mmd`

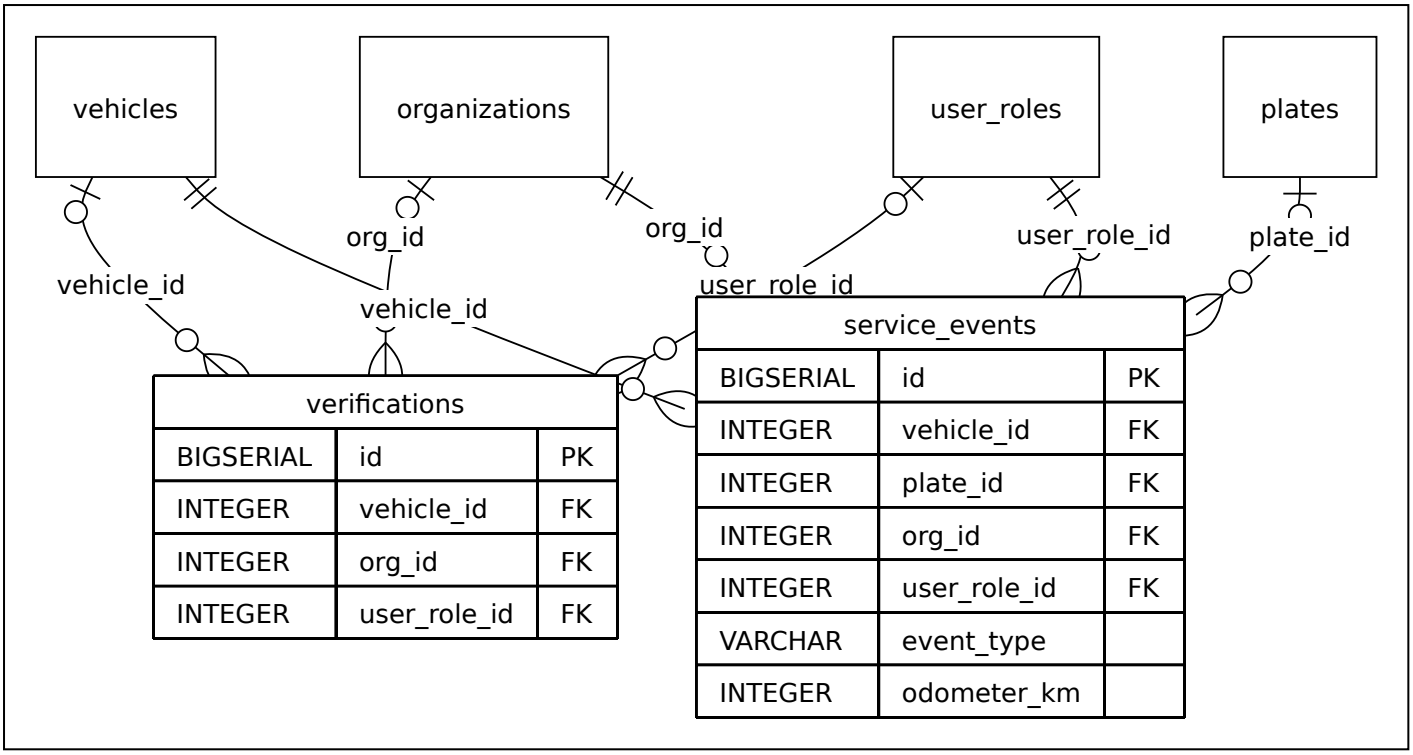


Figure 20 — Modèle de données — vérifications et événements de service

Source : `pdf/views/er-vehicle-4.mmd — vue détaillée de diagrams/er-vehicle.mmd`

`idauto_verifications` fait partie des tables « schéma seulement » (prévues) ; voir l'inventaire.

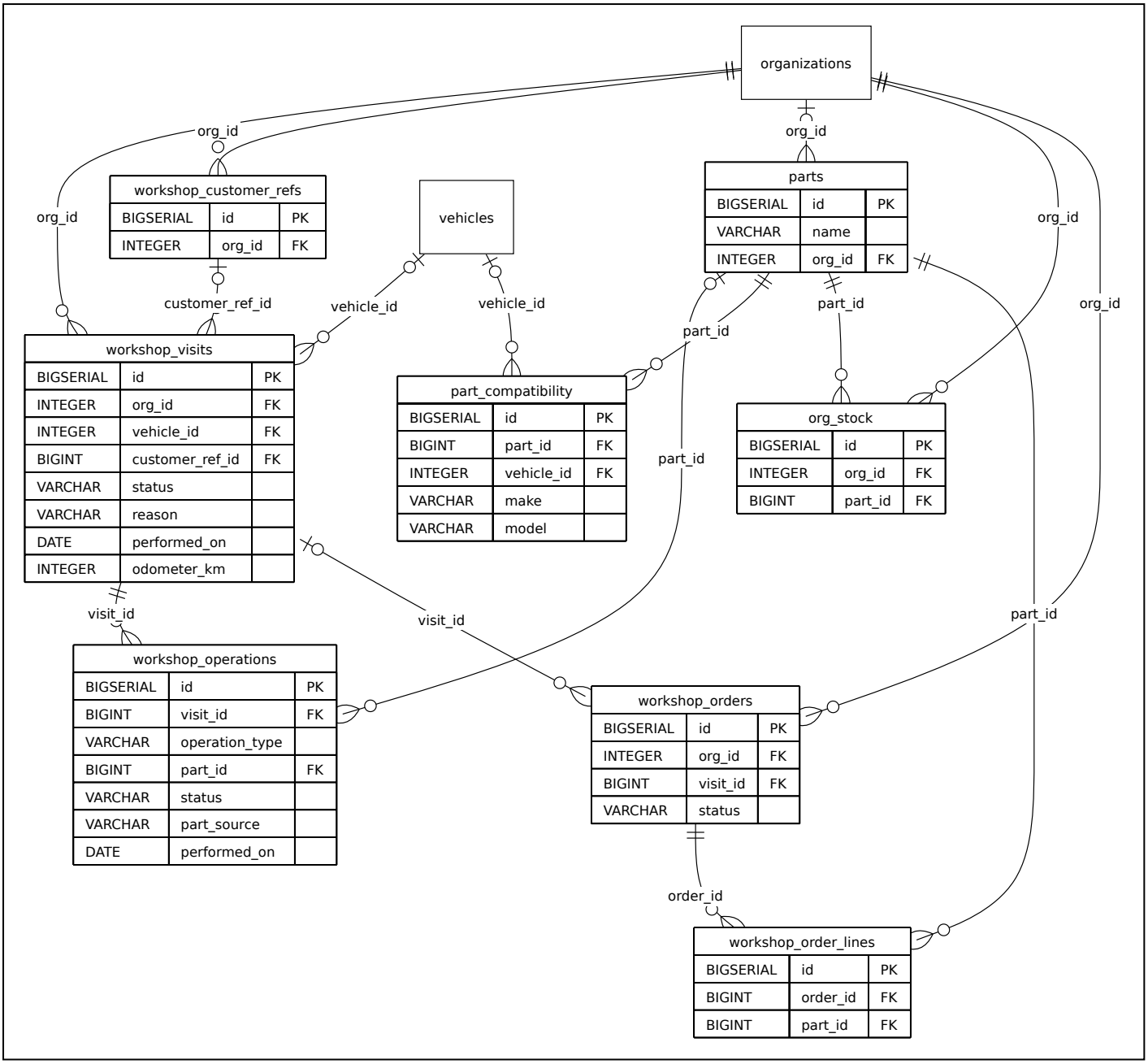


Figure 21 — Modèle de données — entretien en atelier, pièces et provenance

Source : `diagrams/er-workshop.mmd`

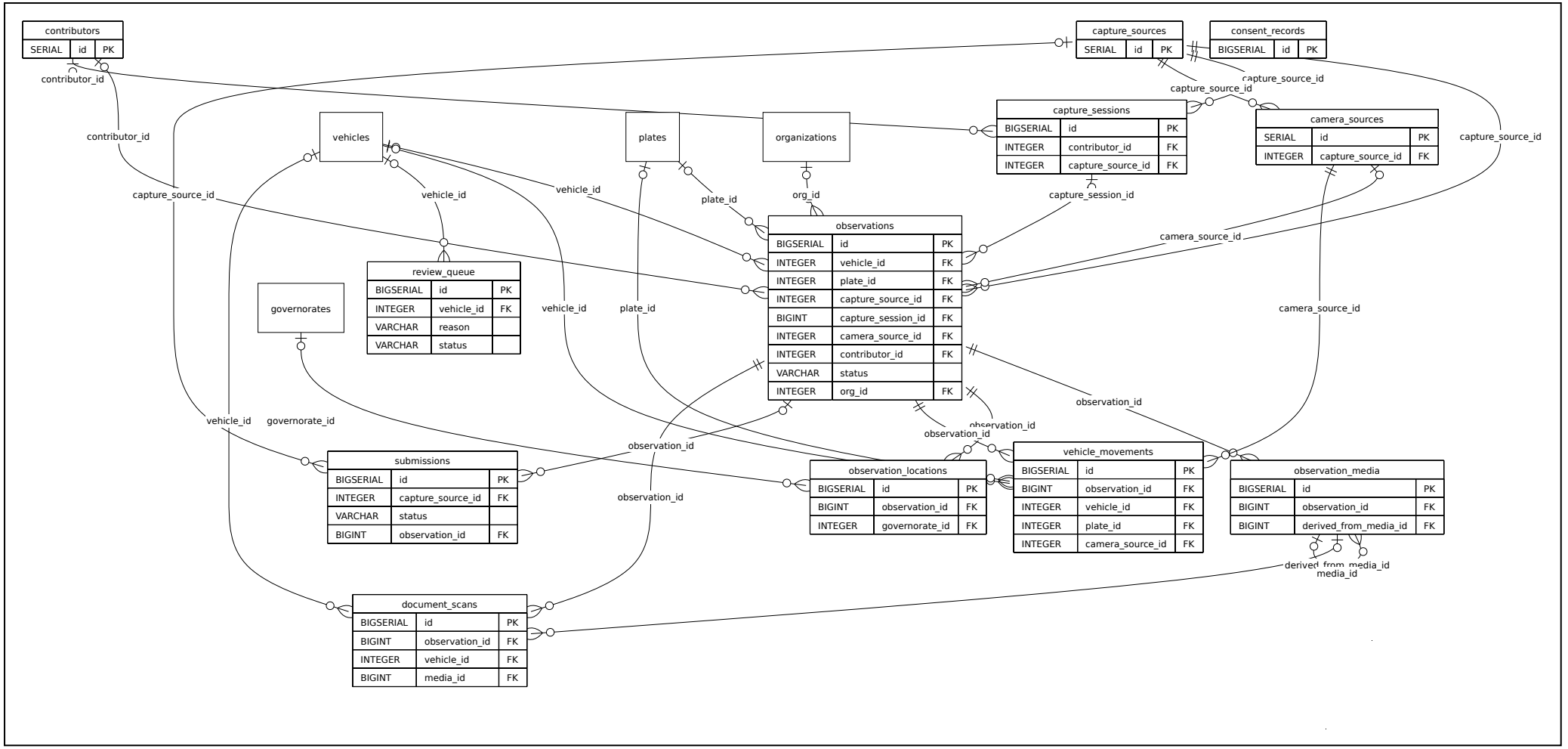


Figure 22 — Modèle de données — capture, réception et revue (vue d'ensemble)

Source : `diagrams/er-capture.mmd`

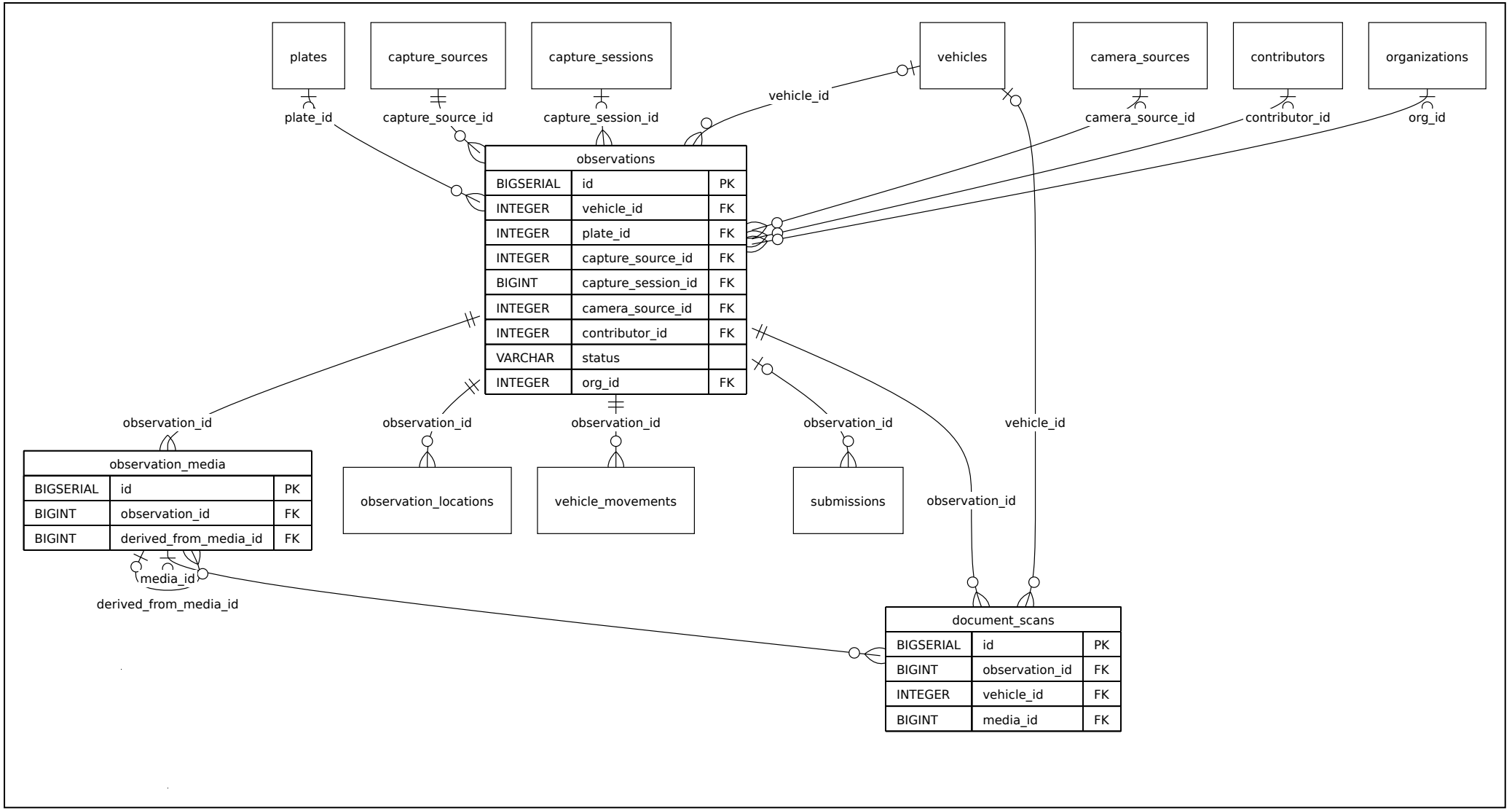


Figure 23 — Modèle de données — observations, médias et scans de documents

Source : `pdf/views/er-capture-1.mmd — vue détaillée de diagrams/er-capture.mmd`

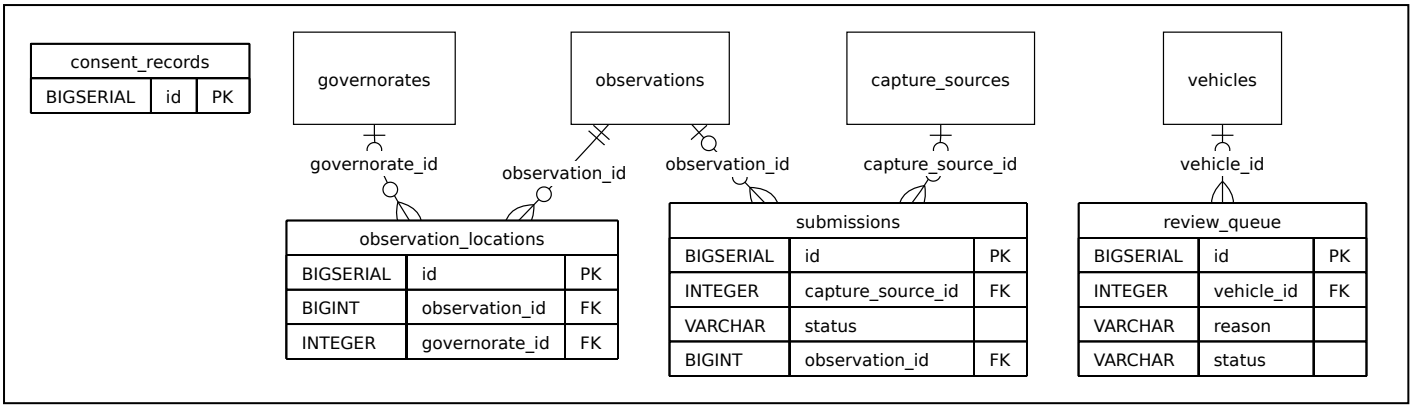


Figure 24 — Modèle de données — localisations, soumissions, file de revue et consentements

Source : `pdf/views/er-capture-2.mmd — vue détaillée de diagrams/er-capture.mmd`

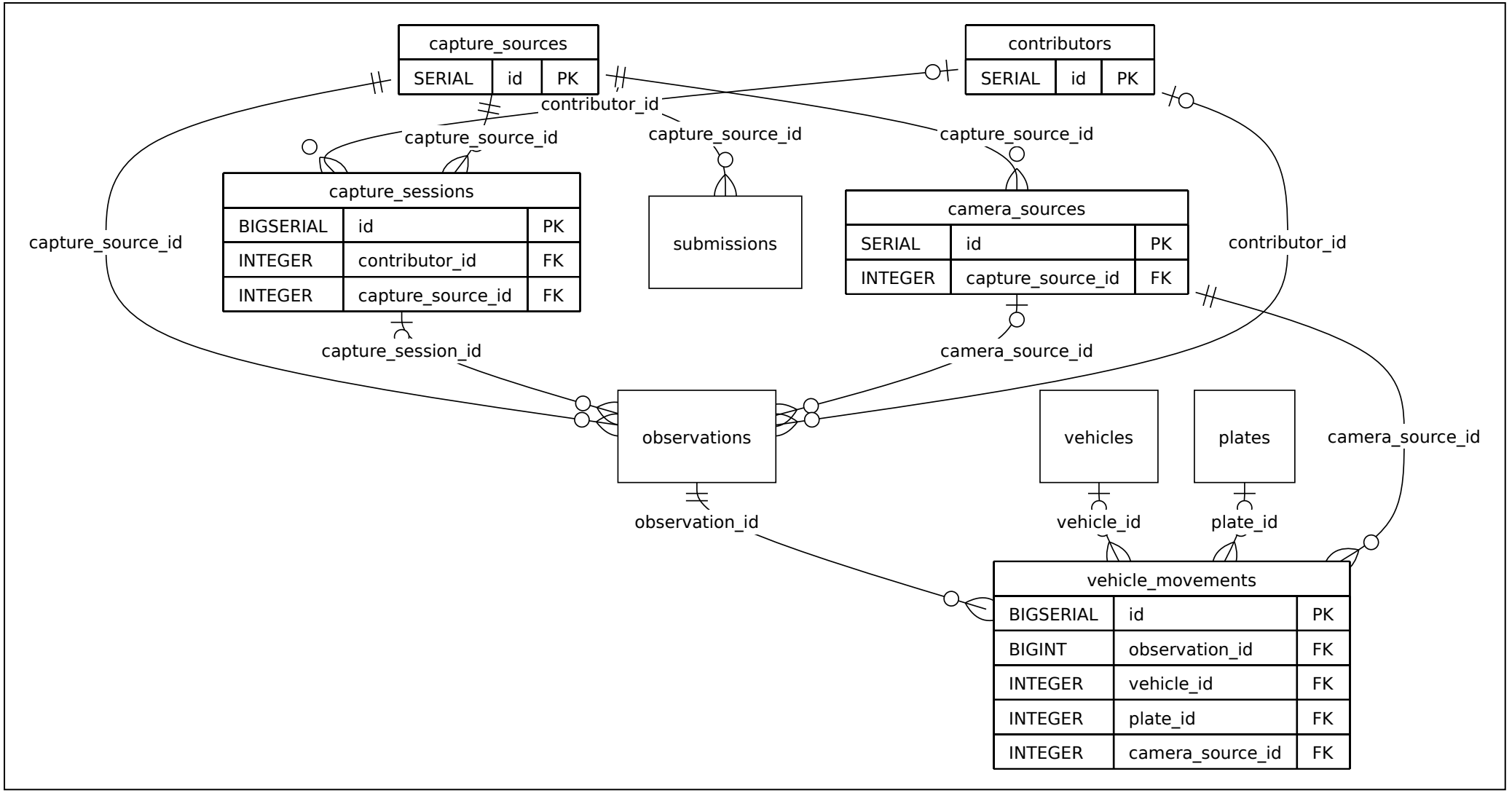


Figure 25 — Modèle de données — sources de capture, sessions, contributeurs, caméras et mouvements

Source : `pdf/views/er-capture-3.mmd — vue détaillée de diagrams/er-capture.mmd`

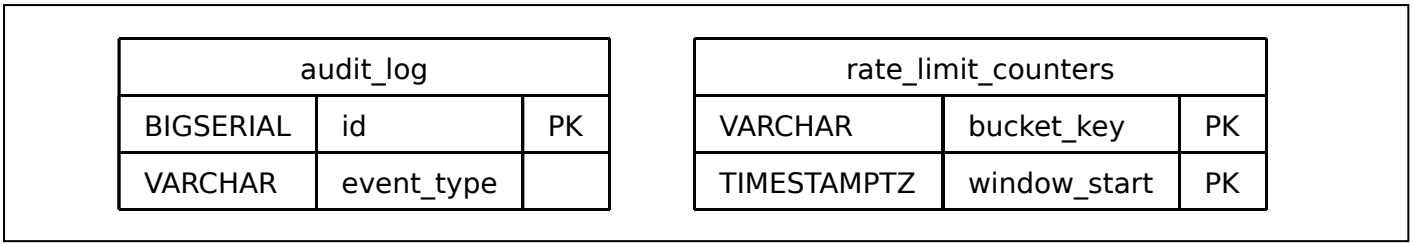


Figure 26 — Modèle de données — éléments transverses : audit et limitation de débit

Source : `diagrams/er-crosscut.mmd`

5.4 Relation décrite seulement par un commentaire

Table.colonne	Cible documentée	Remarque
`idauto_camera_sources.org_id`	`idauto_organizations`	commentaire `-- FK → idauto_organizations` dans `schema.sql` ; aucune contrainte

5.5 Index uniques partiels porteurs de règles métier

Index	Règle
`uidx_idauto_plates_number_active`	une seule plaque **active** par numéro
`uq_owner_contact_current`	un seul contact propriétaire courant par véhicule et par atelier (`COALESCE(org_id, 0)`, `superseded_at IS NULL`)
`uq_vehicle_notes_org`	une note par véhicule et par atelier
`uidx_idauto_parts_ref_brand_org`	une référence de catalogue par marque et par atelier (ou partagée)

6. Architecture logique

IDauto est un **monolithe en couches** : un seul processus Node.js sert les pages, les fichiers statiques et l'API JSON. Il n'y a ni framework web, ni gestionnaire de messages, ni serveur de cache, ni processus de fond. La source du diagramme est `diagrams/logical-architecture.mmd`.

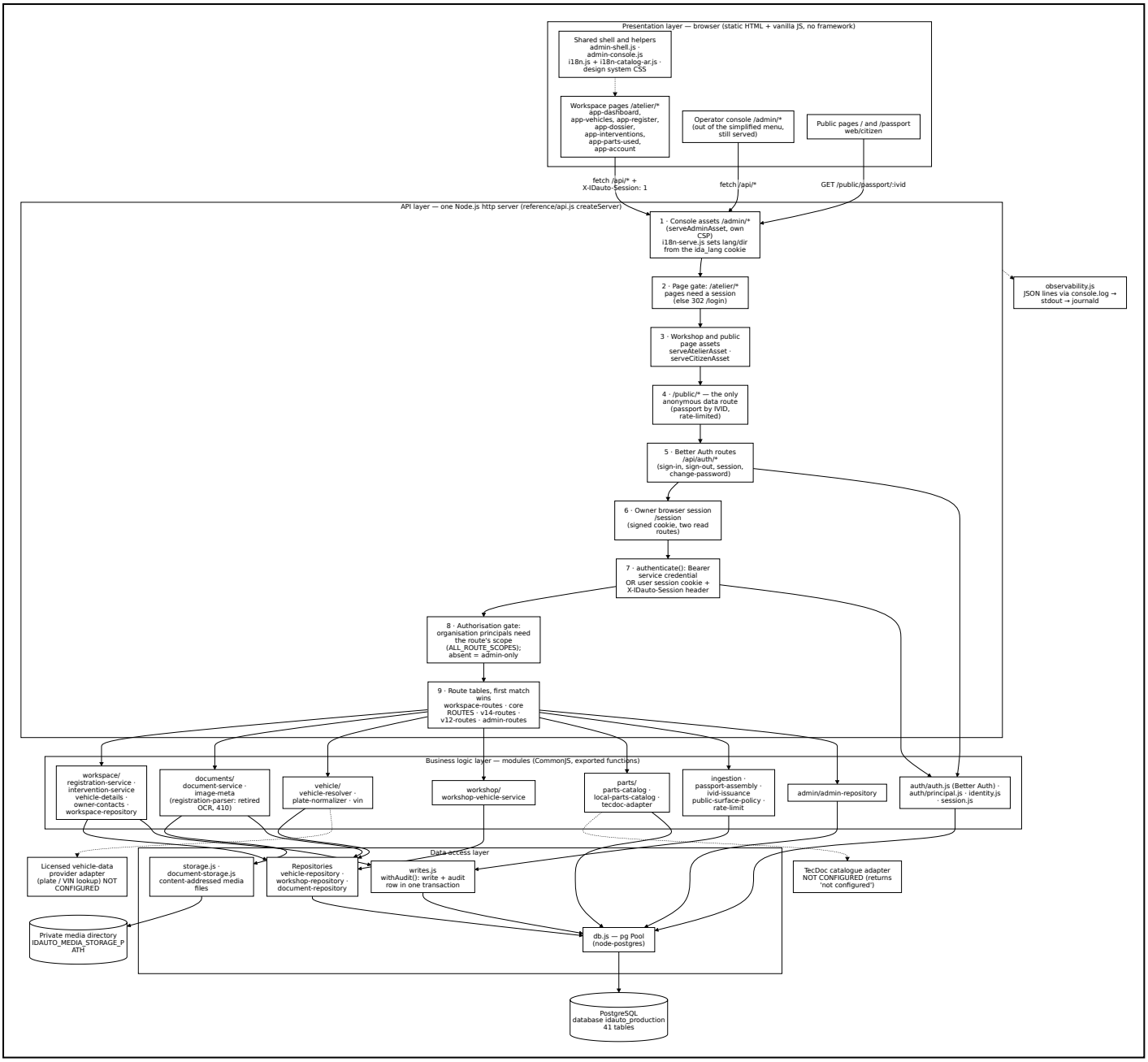


Figure 27 — Architecture logique — couches et chaîne de traitement des requêtes (vue d'ensemble)

Source : `diagrams/logical-architecture.mmd`

- La couche API est dessinée **dans l'ordre où une requête la traverse** : les numéros 1 à 9 suivent l'ordre des contrôles de `createServer()` (`reference/api.js`). La première étape qui répond termine la requête.
- Les flèches en pointillés sont des chemins facultatifs ou secondaires : adaptateurs non configurés, journaux, script commun chargé par chaque page.

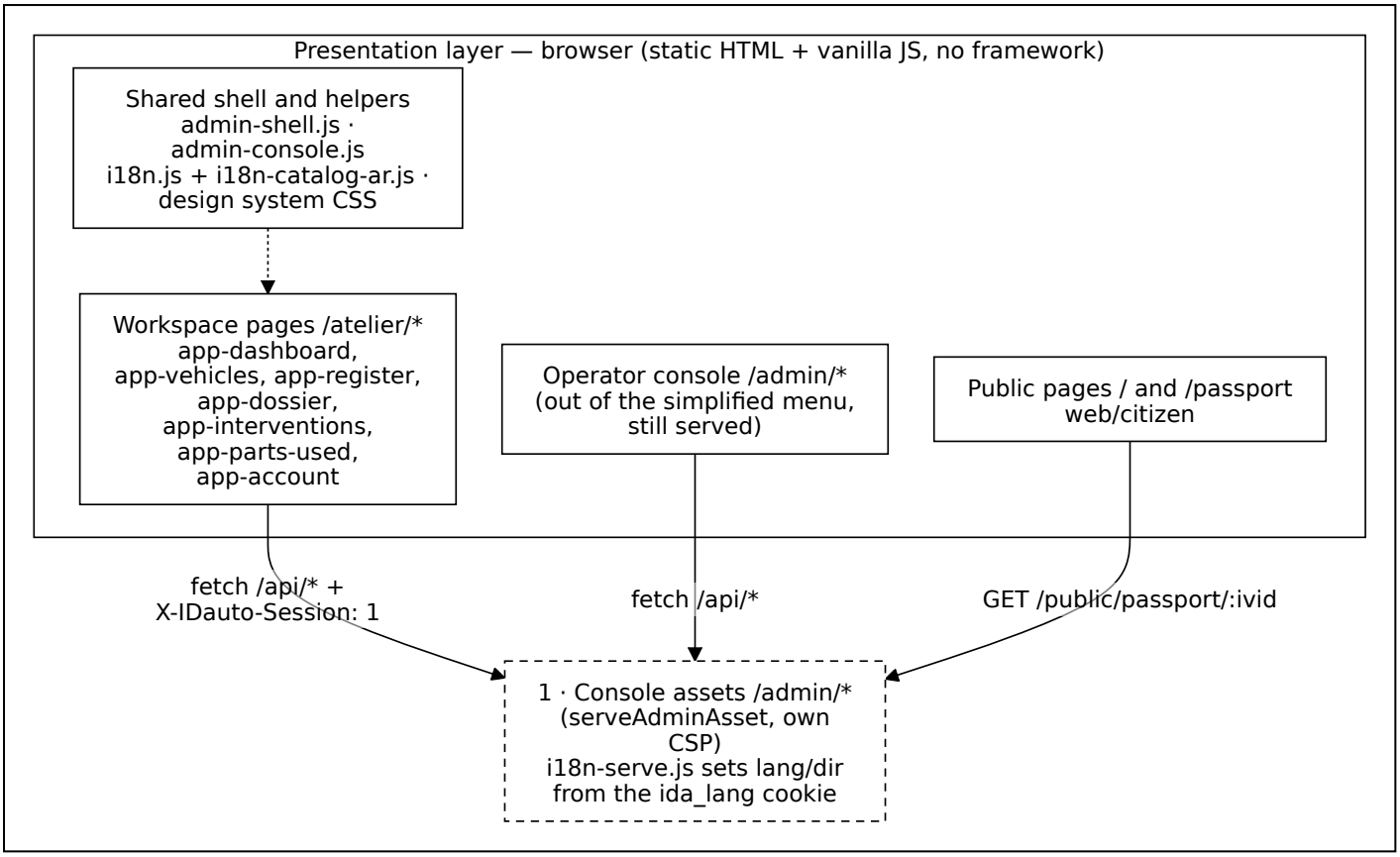


Figure 28 — Architecture logique — couche présentation

Source : `pdf/views/logical-architecture-1.mmd — vue détaillée de diagrams/logical-architecture.mmd`

Les pages ne portent aucune règle métier. Elles appellent l'API avec `fetch` et l'en-tête `X-IDauto-Session: 1`.

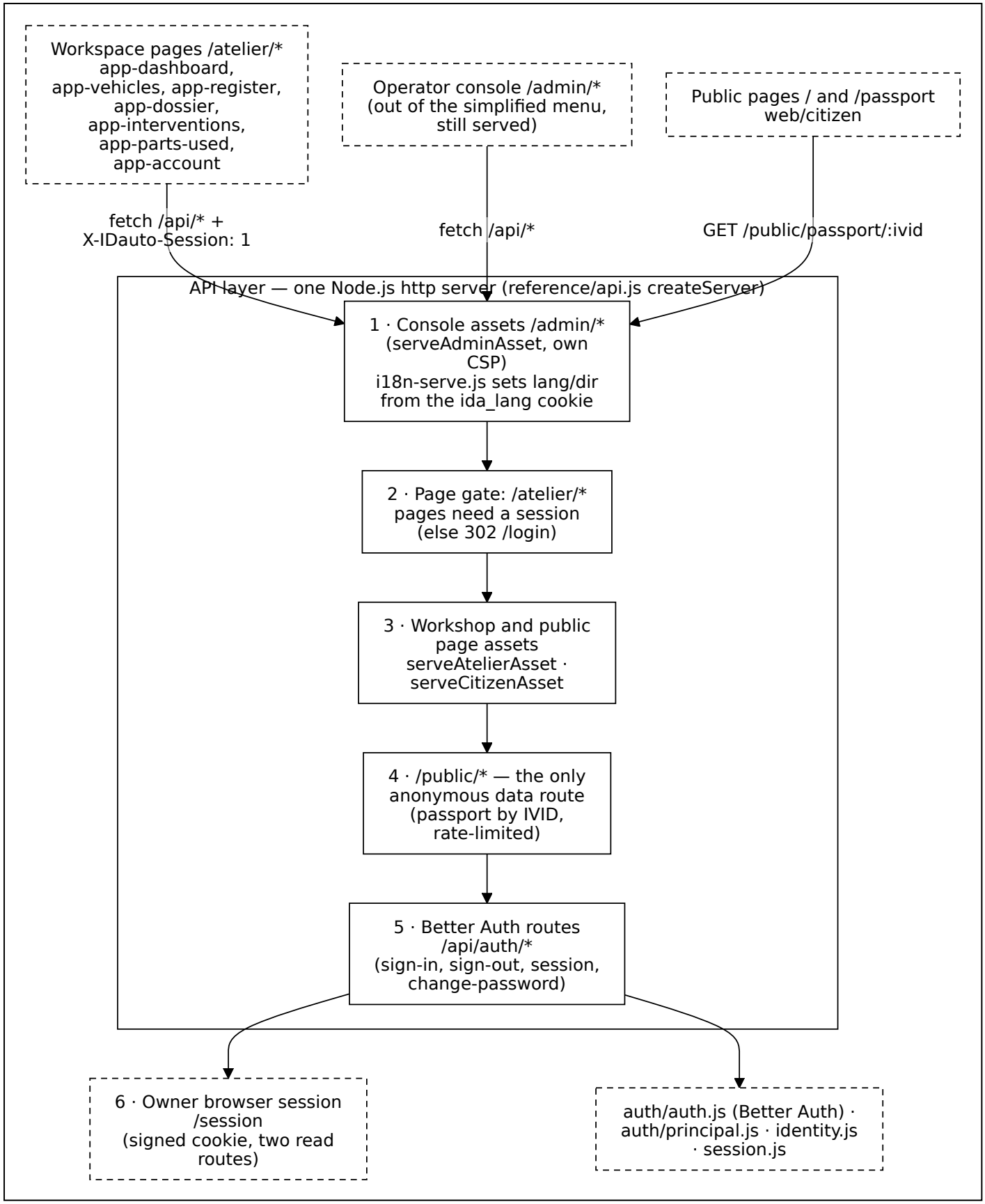


Figure 29 — Architecture logique — chaîne de traitement de l'API, étapes 1 à 5

Source : `pdf/views/logical-architecture-2.mmd — vue détaillée de diagrams/logical-architecture.mmd`

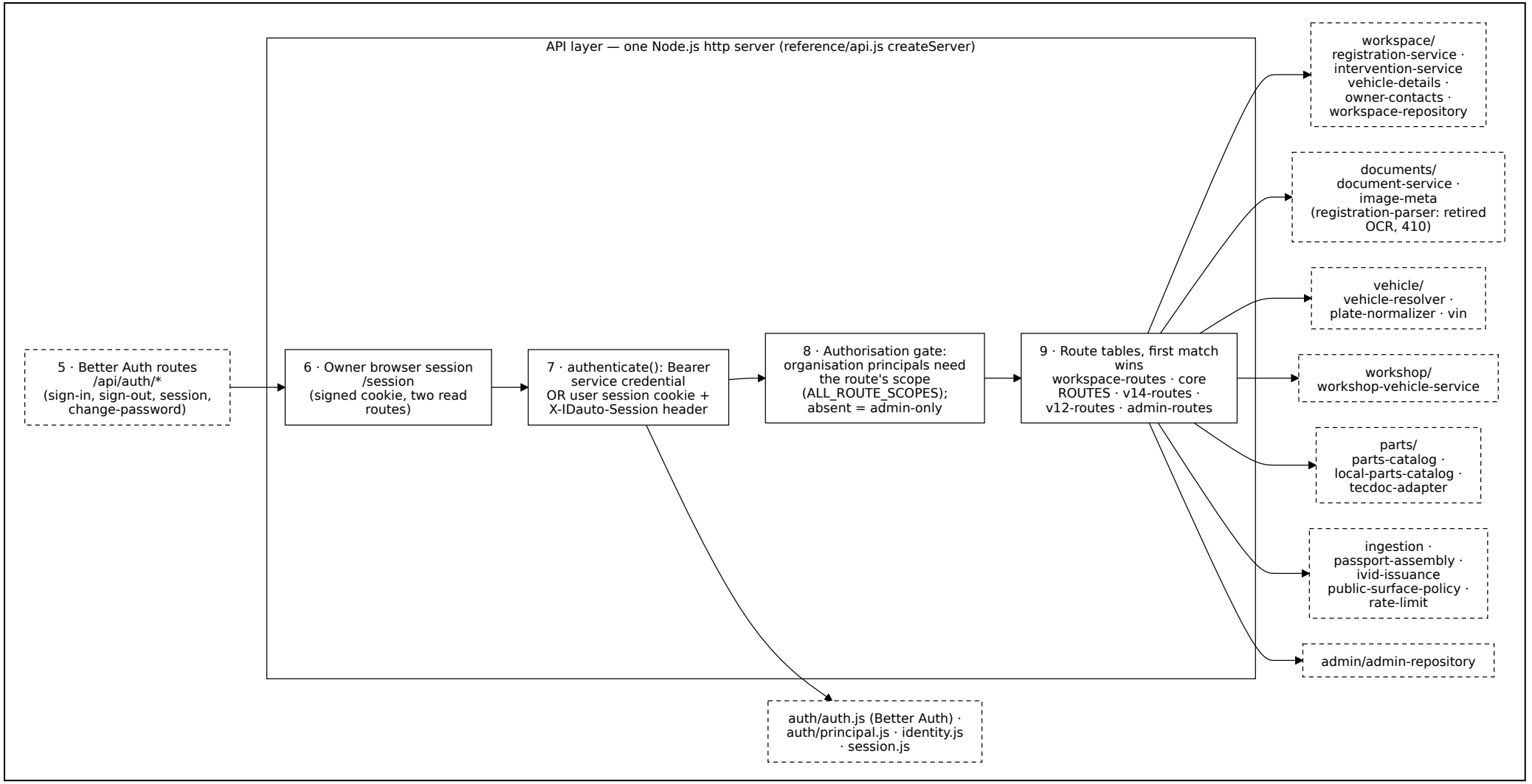


Figure 30 — Architecture logique — étapes 6 à 9 et aiguillage vers les modules métier

Source : `pdf/views/logical-architecture-3.mmd — vue détaillée de diagrams/logical-architecture.mmd`

L'ordre des tables de routes est `workspace`, `ROUTES` (cœur), `v14`, `v12`, `admin`. La première route qui correspond l'emporte.

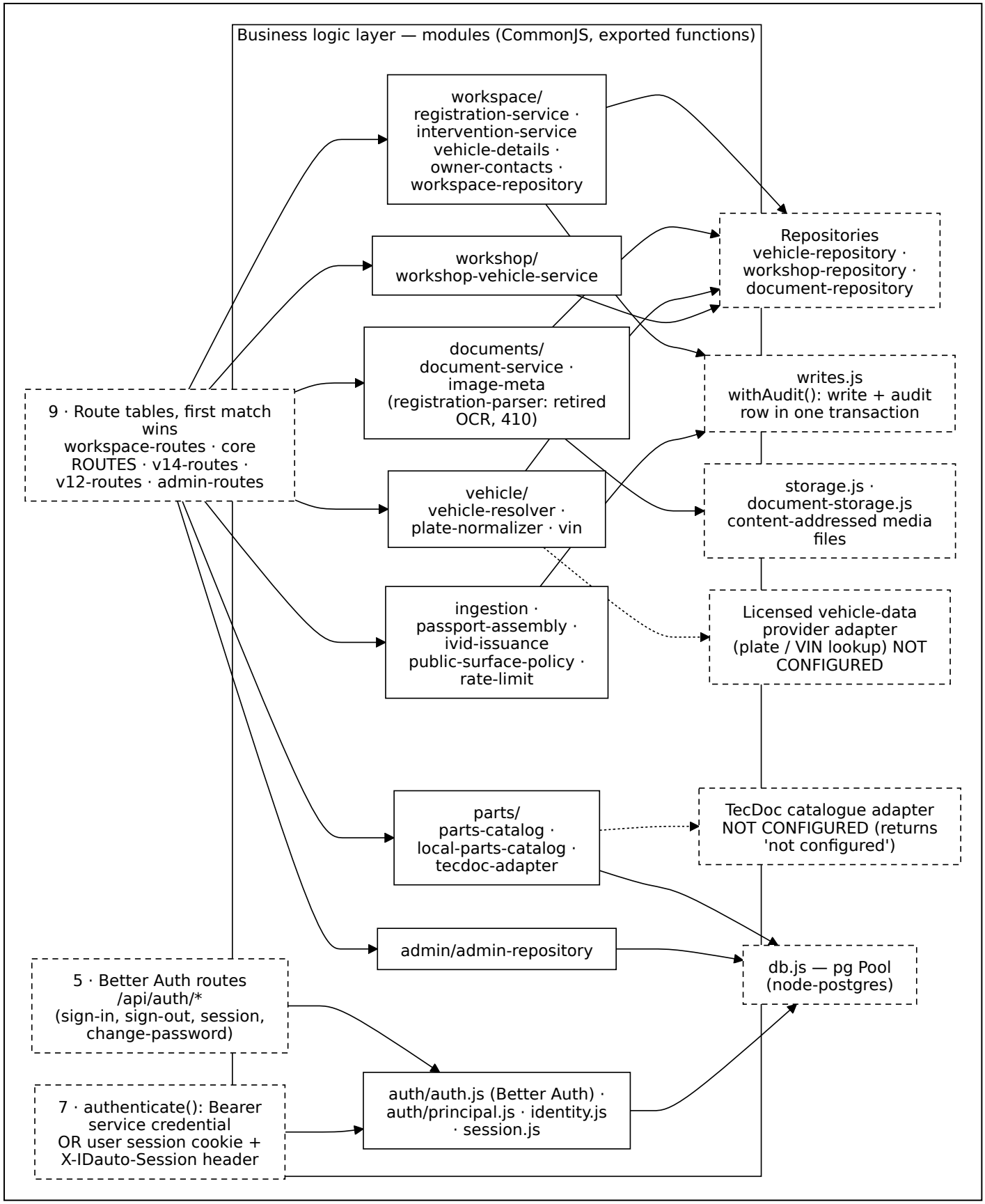


Figure 31 — Architecture logique — couche métier

Source : `pdf/views/logical-architecture-4.mmd — vue détaillée de diagrams/logical-architecture.mmd`

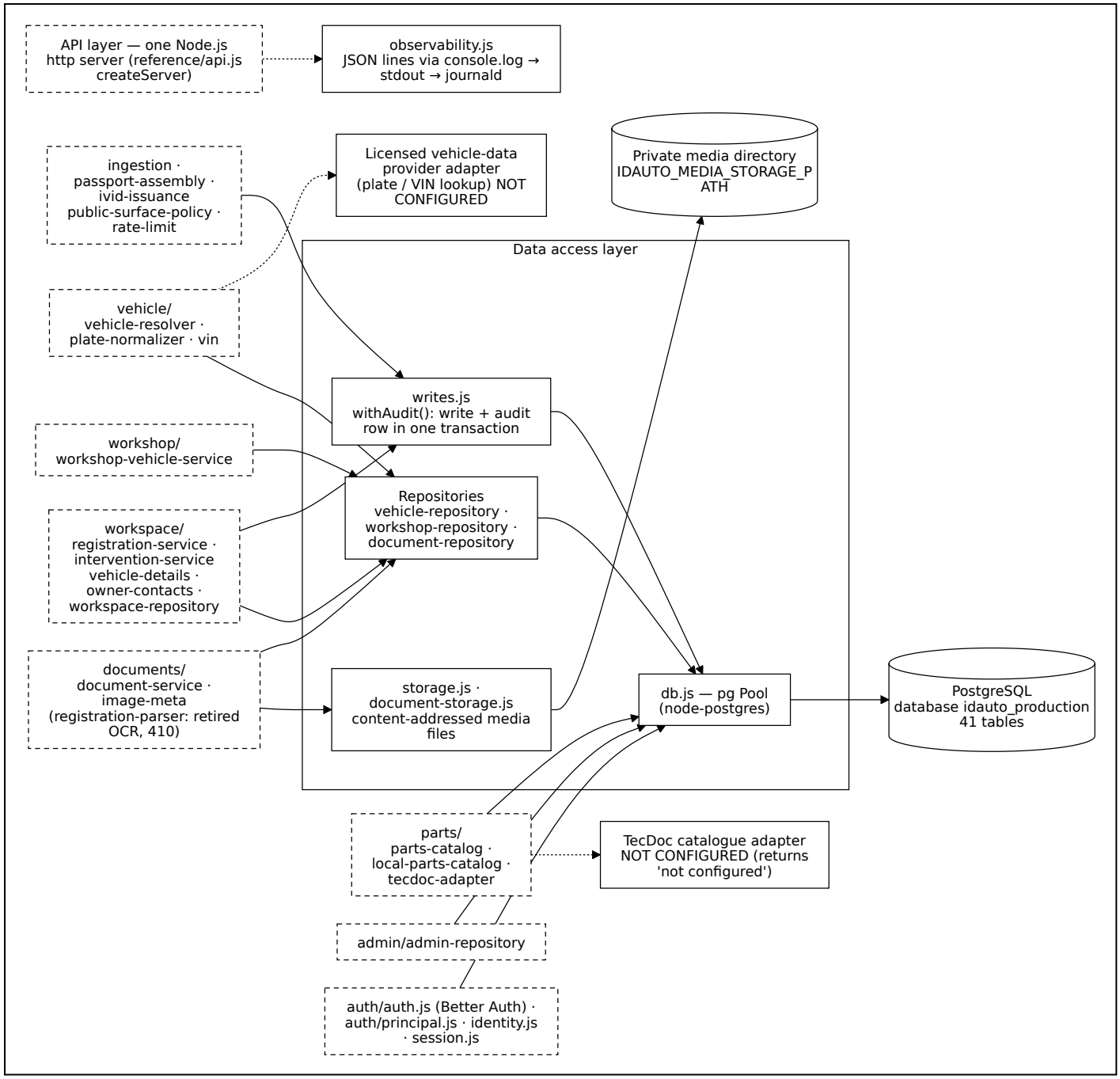


Figure 32 — Architecture logique — accès aux données, stockage, journaux et adaptateurs non configurés

Source : `pdf/views/logical-architecture-5.mmd — vue détaillée de diagrams/logical-architecture.mmd`

Toute écriture passe par `writes.withAudit()`, qui regroupe la modification et sa ligne d'audit dans une même transaction.

6.1 Couches

Couche	Contenu	Règles qu'elle porte
Présentation	Pages HTML statiques produites par `ops/ui/build-pages.py` ; un script par page (`app-*.js`), l'enveloppe commune (`admin-shell.js`), les aides de la console, le catalogue français et arabe, le CSS du système de design	Aucune règle métier. Appels à l'API avec `fetch` et l'en-tête `X-IDauto-Session: 1`.
API	Chaîne de traitement de `api.js` et tables de routes (`workspace-routes`, `ROUTES` du cœur, `v14-routes`, `v12-routes`, `admin-routes`)	Taille des corps de requête, CSP et en-têtes de sécurité, authentification, contrôle des droits, limites de débit, erreurs JSON
Métier	`workspace/*`, `documents/*`, `vehicle/*`, `workshop/*`, `parts/*`, modules de réception, de passeport et d'IVID, `admin/admin-repository`	Validation, cloisonnement par organisation, suppression par archivage seulement, audit des lectures sensibles (contact propriétaire, VIN)
Accès aux données	Dépôts, `writes.js`, `db.js` (une seule réserve `pg`), `storage.js` et `document-storage.js`	Transactions, lignes d'audit, noms de fichiers adressés par contenu
Données	PostgreSQL (41 tables), répertoire privé des médias	Clés étrangères, CHECK, index uniques partiels

6.2 Préoccupations transverses

- **Audit.** `idauto_audit_log` reçoit une ligne par écriture, dans la même transaction. Les lectures sensibles sont aussi auditées.
- **Journalisation.** `observability.js` écrit une ligne JSON par événement (décisions d'authentification, erreurs) sur la sortie standard, que le journal systemd conserve. Il n'y a pas de service externe de journaux ni de métriques.
- **Internationalisation.** Les pages sont écrites en français. `i18n-serve.js` fixe `lang` et `dir` côté serveur d'après le cookie `ida_lang` ; `i18n.js` traduit dans le navigateur à partir de `i18n-catalog-ar.js`.
- **Configuration.** Voir le chapitre « Infrastructure, sauvegardes et environnements ». Seuls les *noms* des variables sont documentés.

7. Diagramme de séquence

Ce diagramme suit l'enregistrement d'une intervention d'entretien (`POST /api/workspace/interventions`). C'est le chemin d'écriture le plus complet de l'application simplifiée, et chacune de ses étapes est dans le code (`workspace-routes.js postIntervention`, `intervention-service.js create`, `writes.js withAudit`). La source est `diagrams/sequence-intervention.mmd`, une copie à l'identique du diagramme de `04-logical-architecture.md`.

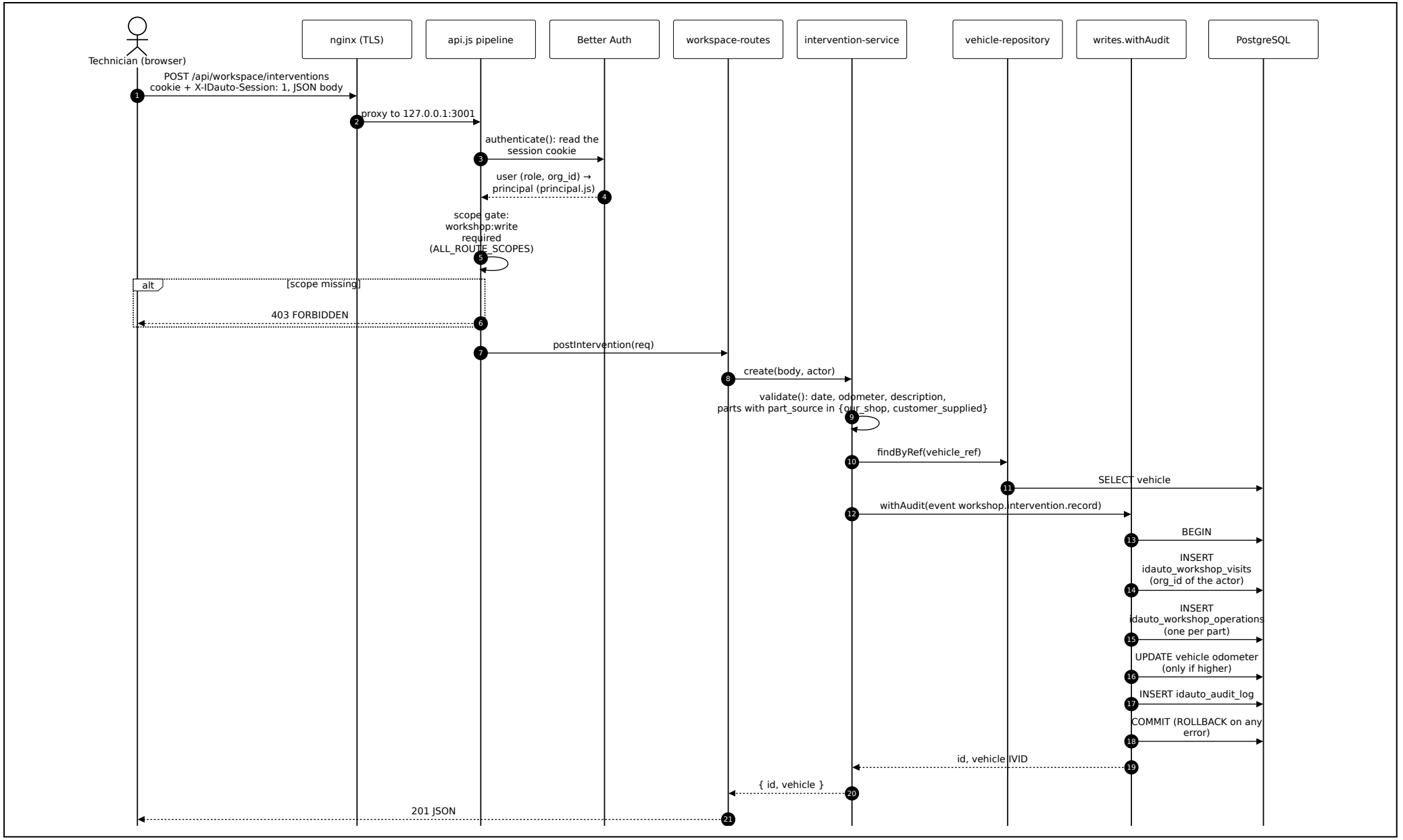


Figure 33 — Séquence — enregistrement d'une intervention d'entretien (vue d'ensemble)

Source : `diagrams/sequence-intervention.mmd (copie de 04-logical-architecture.md)`

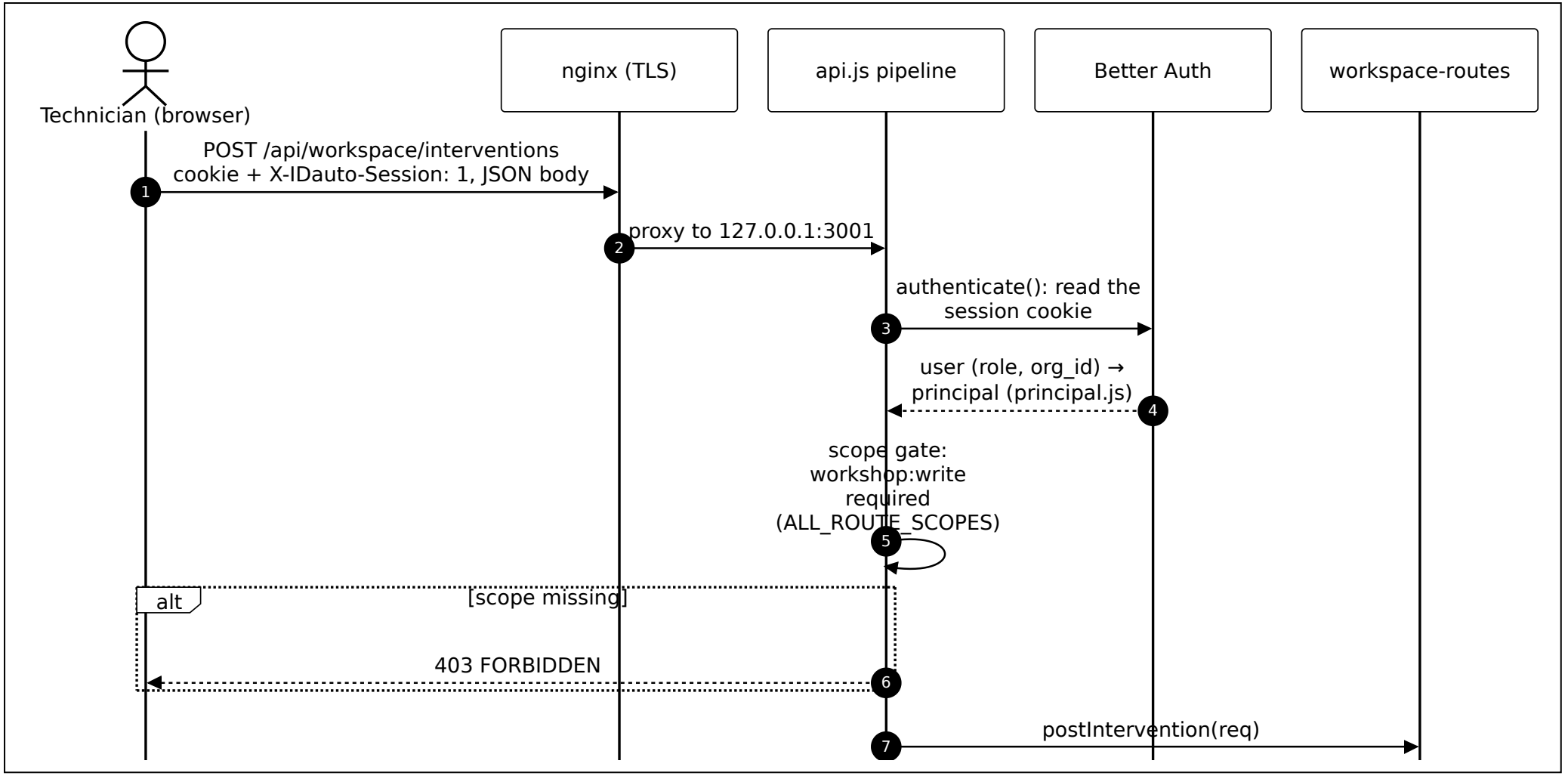


Figure 34 — Séquence — phase 1 : authentification et contrôle des droits (messages 1 à 7)

Source : `pdf/views/sequence-intervention-1.mmd — vue détaillée de diagrams/sequence-intervention.mmd`

Si le droit `workshop:write` manque, la requête s'arrête ici avec une réponse 403.

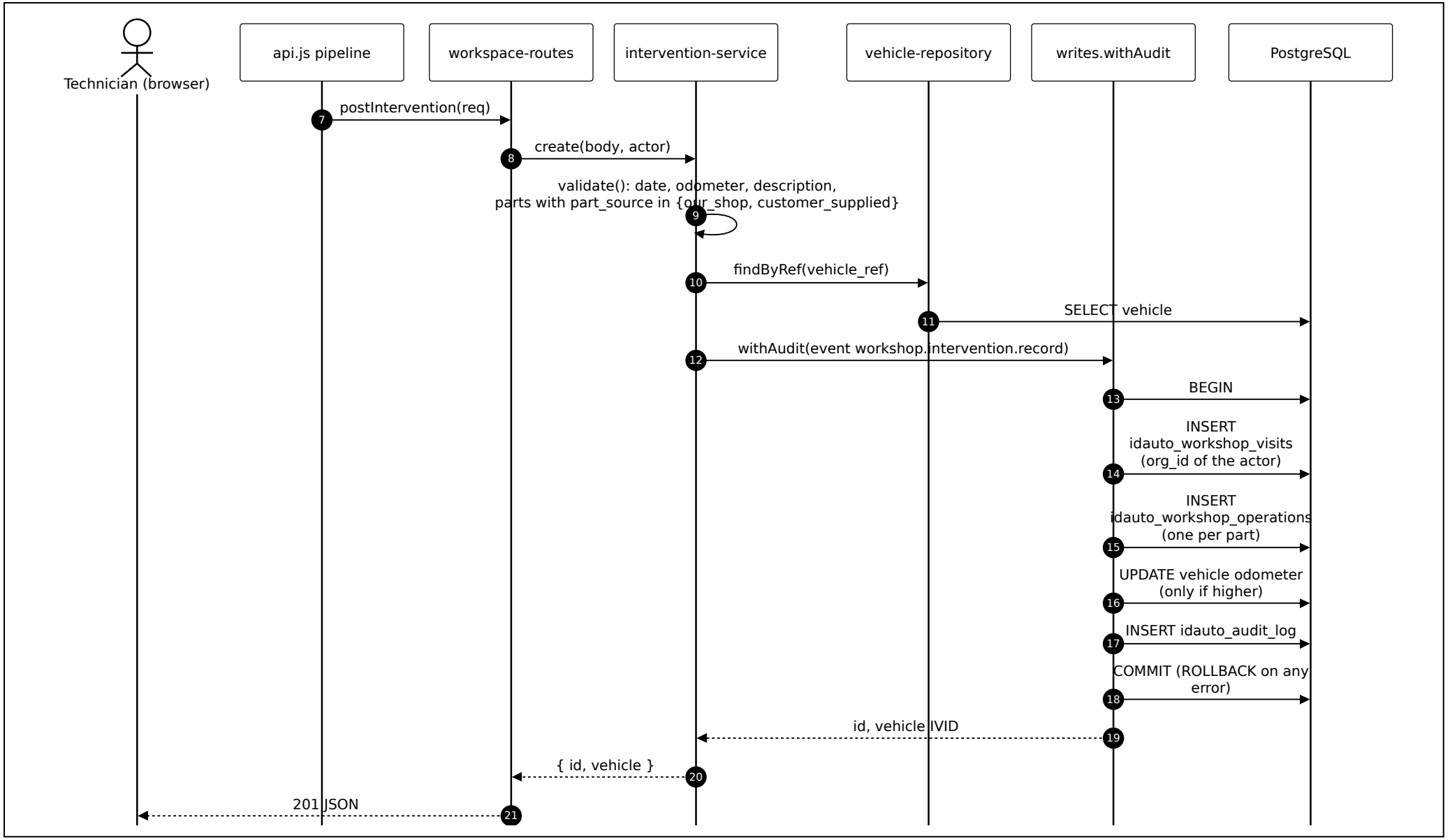


Figure 35 — Séquence — phase 2 : service métier et transaction (messages 7 à 21)

Source : `pdf/views/sequence-intervention-2.mmd — vue détaillée de diagrams/sequence-intervention.mmd`

Le message 7 est repris en tête de la phase 2 pour montrer l'enchaînement ; la numérotation continue celle de la phase 1.

7.1 Lecture

- Une erreur de validation (400), un véhicule inconnu (404) ou une organisation manquante (403) arrêtent le traitement avant le début de la transaction.
- Une erreur pendant la transaction annule ensemble la visite, ses opérations et la ligne d'audit.
- `withAudit` refuse de s'exécuter sans identité authentifiée (401) : une écriture non attribuée est donc impossible.
- Le kilométrage du véhicule n'est mis à jour que si la nouvelle valeur est plus élevée.

8. Architecture physique et déploiement

Méthode. Les valeurs de production viennent de vérifications en lecture seule faites le 10 novembre 2026 sur le serveur de production :

- état des services et versions des programmes ;
- ports en écoute et site nginx ;
- liste des conteneurs ;
- comptage des tables de la base.

La configuration a été lue sous forme de **noms et de nombres de variables** seulement, jamais de valeurs.

Volontairement omis :

- adresses IP, noms d'hôtes privés, chemins du système de fichiers ;
- noms de comptes, jetons et clés ;
- le compartiment de sauvegarde et les identifiants de son fournisseur.

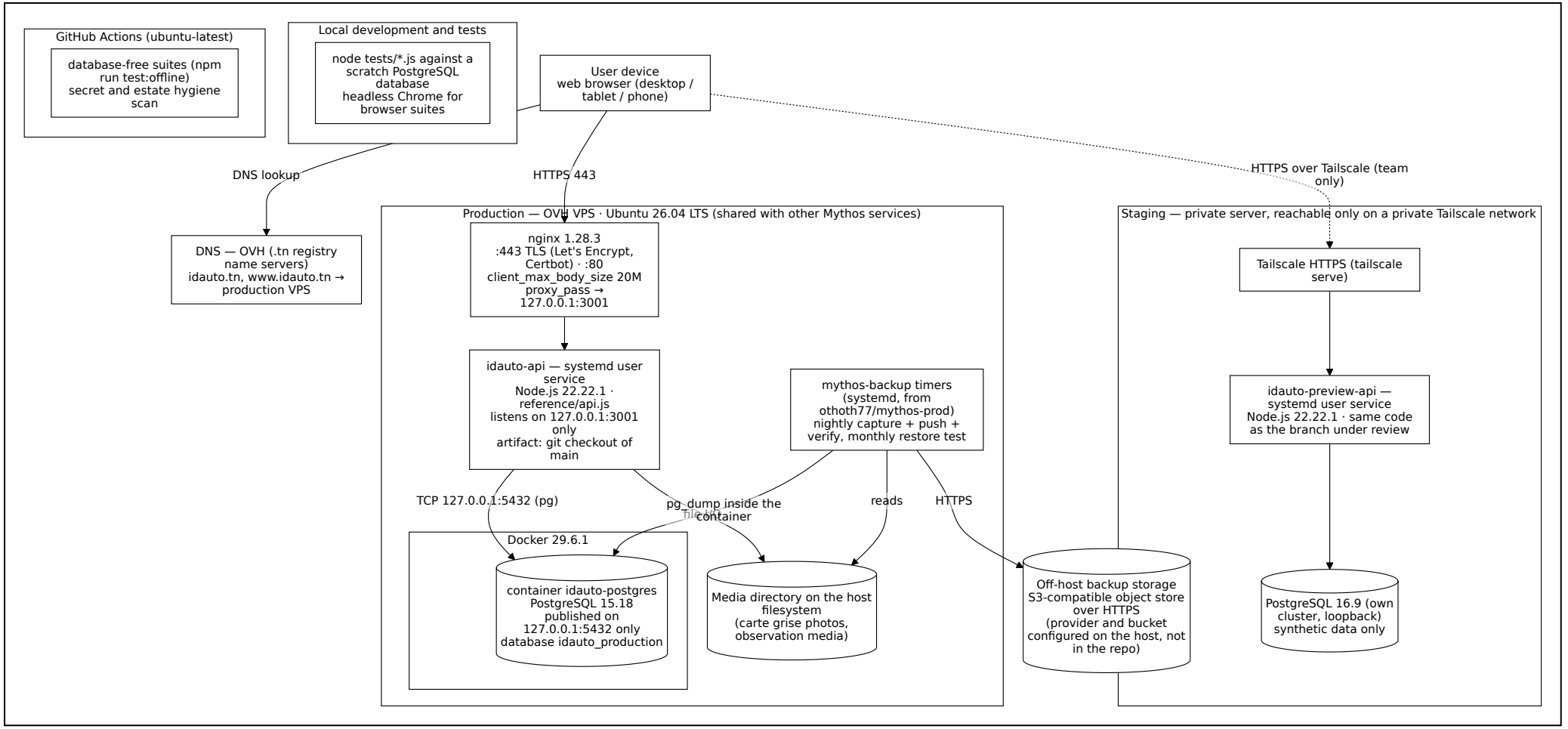


Figure 36 — Architecture physique — nœuds de production, de pré-production, de développement local et d'intégration continue

Source : `diagrams/physical-architecture.mmd`

- Les flèches pleines sont les chemins réseau ou fichiers utilisés en fonctionnement normal.
- La flèche en pointillés est l'accès privé à la pré-production, réservé aux membres de l'équipe présents sur le réseau privé.
- Seul nginx est joignable depuis Internet. Le processus Node et PostgreSQL n'écoutent que sur l'interface locale.

8.1 Production

Nœud	Ce qui y tourne	Faits constatés
DNS	serveurs de noms OVH pour `idauto.tn`	`idauto.tn` et `www.idauto.tn` pointent vers le serveur de production
Serveur de production	serveur privé virtuel OVH, Ubuntu 26.04 LTS. Il est partagé avec d'autres services du propriétaire (Mythos), qui se partagent donc son disque et sa mémoire.	`lsb_release`, `systemctl`
nginx 1.28.3	TLS sur le port 443 (Let's Encrypt par Certbot), HTTP sur le port 80, `client_max_body_size 20M`, proxy inverse vers l'API sur l'interface locale	`nginx -v`, configuration du site
idauto-api	service systemd **utilisateur**, Node.js 22.22.1, exécute `reference/api.js` depuis une copie Git de `main` ; redémarré par systemd	`systemctl --user status`, `node --version`, `ss -ltn`
PostgreSQL 15.18	conteneur Docker 29.6.1 `idauto-postgres`, publié sur l'interface locale uniquement ; base **`idauto_production`** avec les 41 tables du dépôt et aucune autre	`SELECT version()`, `docker ps`, comparaison des listes de tables
Répertoire des médias	système de fichiers du serveur ; photos de carte grise et médias d'observation, adressés par contenu	`IDAUTO_MEDIA_STORAGE_PATH` est défini (valeur non affichée)
Minuteries de sauvegarde	minuteries systemd du dépôt `othoth77/mythos-prod` pour la chaîne IDauto (voir le chapitre suivant)	`systemctl list-timers`, `systemctl show … ExecStart`
Stockage hors serveur	stockage objet compatible S3 en HTTPS, alimenté par `ops/offhost-backup.js`	adaptateur `ops/adapters/s3-compatible.js`

Chemin d'une requête. Navigateur → DNS → nginx sur le port 443 (TLS) → port local de l'API → réserve `pg` → PostgreSQL dans Docker. Les fichiers médias sont lus et écrits sur le système de fichiers du serveur.

Ce qui n'existe pas en production :

- ni répartiteur de charge, ni seconde instance de l'application ;
- ni réseau de diffusion de contenu (CDN), ni serveur de cache, ni file de messages ;
- aucun service externe de journaux ou de métriques : les journaux vont au journal systemd ;
- aucune configuration TecDoc ni de fournisseur de données véhicule ;
- aucun service de courriel.

8.2 Procédure de déploiement pratiquée

1. Mettre à jour la copie Git du serveur de production sur le commit fusionné de `main`.
2. Appliquer explicitement toute nouvelle migration avec `psql -d idauto_production`.
3. Redémarrer le service utilisateur.
4. Exécuter un contrôle d'acceptation sur `https://idauto.tn`.

Le retour arrière consiste à revenir au commit précédent puis à redémarrer le service. L'application n'a ni étape de construction ni image de conteneur.

9. Sécurité, authentification et permissions

9.1 Authentification

Principal	Comment il est obtenu	Droits	Portée des données
Administrateur	compte web avec `role = 'admin'`, ou identité de service par jeton associée à l'administration	`*` (tous)	toutes les organisations
Gestionnaire	compte web avec `role = 'manager'` et un `org_id`	24 droits : les 17 du technicien plus 7	sa propre organisation
Technicien	compte web avec `role = 'technician'` et un `org_id`	17 droits	sa propre organisation
Client de service d'une organisation	jeton porteur listé dans `IDAUTO_ADMIN_IDENTITIES`, avec un `org_id` et des droits explicites	selon la configuration	sa propre organisation
Session navigateur du propriétaire	cookie signé installé depuis la console (`/session/enroll`) quand `IDAUTO_SESSION_SECRET` est défini	deux routes de lecture seulement	—
Anonyme	aucun	`/public/*` seulement (passeport par IVID), connexion, pages publiques	—

Better Auth gère les comptes par courriel et mot de passe (empreinte scrypt) et les sessions par cookie. La connexion et le changement de mot de passe sont limités à 5 tentatives par minute et par client.

9.2 Rôles et droits

Les droits sont définis dans `reference/auth/principal.js`. Ceux du technicien forment un sous-ensemble strict de ceux du gestionnaire.

Rôle	Droits
Technicien (17)	`vehicle:read`, `vehicle:write`, `vehicle:search`, `vehicle:resolve`, `plate:read`, `fact:read`, `observation:read`, `observation:write`, `passport:read`, `identity:resolve`, `parts:read`, `workshop:read`, `workshop:write`, `document:read`, `document:write`, `owner:read`, `owner:write`
Gestionnaire (24)	les 17 droits du technicien, plus `vin:search`, `fact:write`, `parts:write`, `stock:write`, `document:delete`, `team:read`, `vehicle:delete`
Administrateur	`*` — toutes les routes, toutes les organisations

Règle d'autorisation (`api.js`, après `authenticate()`) :

- un principal d'organisation doit détenir le droit que `ALL_ROUTE_SCOPES` associe à la route ;
- une route absente de cette table est **réservée à l'administrateur** ;
- le cloisonnement par organisation est appliqué une seconde fois **en SQL** par chaque requête de l'espace de travail (`org_id = $n`).

9.3 Frontières de sécurité

Frontière	Mécanisme	Source
Internet → serveur	nginx termine le TLS. Le processus Node n'écoute que sur `127.0.0.1`.	`api.js`, `listen(port, '127.0.0.1')` ; site nginx
Anonyme → données	Seul `/public/passport/:ivid` est ouvert (IVID uniquement, débit limité, désactivable par configuration). La recherche par plaque est fermée.	`api.js`, `public-surface-policy.js`
Pages	Les pages `/atelier/*` redirigent vers `/login` sans session.	contrôle des pages dans `api.js`
API	Sans session (ou sans jeton porteur) : 401. Les requêtes par cookie exigent en plus l'en-tête `X-IDauto-Session: 1`, qui protège contre les requêtes intersites (CSRF).	`api.js authenticate()`

Frontière	Mécanisme	Source
Rôle	Droit de la route lu dans `ALL_ROUTE_SCOPES` ; une route absente est réservée à l'administrateur	`api.js`, `principal.js`
Organisation	`org_id = $n` dans chaque requête de l'espace de travail ; contacts propriétaires et notes privés à chaque atelier	`workspace/*.js`
Lectures sensibles	Chaque lecture d'un contact propriétaire et chaque divulgation d'un VIN écrivent une ligne d'audit	`owner-contacts.js`, `writes.recordVinDisclosureAudit`
Médias	Photos servies uniquement par des routes authentifiées, après contrôle de l'organisation ; aucune URL publique	`v14-routes.js`, `document-service.image`
Navigateur	Politique de sécurité du contenu (CSP) par famille de pages, `X-Content-Type-Options`, interdiction de l'affichage en cadre	serveurs de fichiers de `api.js`
Force brute	Connexion et changement de mot de passe : 5 par minute et par client	règles de débit de `auth/auth.js`

9.4 Audit et données sensibles

- Chaque écriture est jumelée à une ligne de `idauto_audit_log` dans la même transaction. `withAudit` refuse toute écriture sans identité authentifiée.
- Le VIN n'est renvoyé qu'avec le droit `vin:search`, et chaque divulgation est auditée.
- Le contact du propriétaire est privé à l'atelier qui l'a saisi, et chaque lecture est auditée.
- Le dépôt est public : il ne contient ni identifiant, ni clé, ni jeton. Ce dossier ne cite que des noms de variables, jamais leurs valeurs.

10. Infrastructure, sauvegardes et environnements

10.1 Environnements

Environnement	Faits
Production	serveur privé virtuel OVH partagé ; nginx, service utilisateur `idauto-api`, PostgreSQL 15.18 dans Docker, médias sur le disque du serveur (voir le chapitre précédent)
Pré-production	serveur privé, joignable uniquement par un réseau privé Tailscale (`tailscale serve`, HTTPS), non exposé à Internet. Service utilisateur `idauto-preview-api`, Node.js 22.22.1, qui exécute la branche en revue. PostgreSQL 16.9 dans sa propre instance locale, **données fictives uniquement**. C'est là que le propriétaire valide une modification avant son déploiement.
Développement local	`node tests/*.js` sur une base PostgreSQL jetable ; Chrome sans interface pour les suites navigateur (`ida-ui-browser`)
Intégration continue	GitHub Actions sur `ubuntu-latest`, Node 22 (`.github/workflows/ci.yml`), avec deux tâches : les suites sans base de données (`npm run test:offline`, 16 fichiers) et la tâche « secret and estate hygiene » (`.github/scripts/secret_scan.sh`)

10.2 Sauvegardes

La sauvegarde n'est pas faite par l'application. Des minuteries systemd, installées depuis le dépôt `othoth77/mythos-prod` (`ops/backup/mythos-backup-run.sh`), s'en chargent pour la seule chaîne IDauto :

Minuterie	Fréquence	Rôle
`mythos-backup`	chaque jour, 03 h 30 UTC	capture de `idauto_production` (`pg_dump` exécuté dans le conteneur) et des médias, puis envoi hors du serveur
`mythos-backup-verify`	chaque jour, 15 h 00 UTC	vérification de l'envoi
`mythos-restore-test`	le 1er de chaque mois, 05 h 00 UTC	test de restauration

Les autres minuteries de sauvegarde du serveur appartiennent à d'autres services. Les copies partent vers un stockage objet compatible S3, en HTTPS, par `ops/offhost-backup.js`.

10.3 Observabilité

Chaque événement produit une ligne JSON sur la sortie standard (`observability.js`), que le journal systemd conserve. Il n'y a pas de service externe de journaux ni de métriques.

10.4 Configuration

- **Variables d'environnement utilisées :** `IDAUTO_DB_*`, `IDAUTO_MEDIA_STORAGE_PATH`, `IDAUTO_ADMIN_IDENTITIES`, `IDAUTO_AUTH_SECRET`, `IDAUTO_AUTH_BASE_URL`, `IDAUTO_SESSION_SECRET`, `IDAUTO_PUBLIC_RESOLUTION_CONFIG_PATH`.
- **Variables facultatives, absentes en production** (vérifié par comptage des noms) : `IDAUTO_TECDOC_*`, `IDAUTO_VEHICLE_PROVIDER_*`, `IDAUTO_CG_OCR_ENABLED`, `IDAUTO_ALLOW_MOCK_PROVIDERS`.
- **Fichiers lus :** `config/idauto.example.json` pour les formats de plaque. L'interrupteur de la résolution publique vient du fichier désigné par `IDAUTO_PUBLIC_RESOLUTION_CONFIG_PATH`.

11. Traçabilité entre diagrammes et code source

11.1 Diagrammes → fichiers sources → code

Fig.	Figure	Fichier source	Code ou données de référence
1	Cas d'utilisation implémentés — vue d'ensemble	`diagrams/use-case.mmd (+ use-case.puml)`	reference/auth/principal.js, workspace-routes.js, v14-routes.js, api.js, ops/auth-users.js
2	Cas d'utilisation — authentification, consultation, administration et API machine	`pdf/views/use-case-1.mmd`	reference/auth/principal.js, workspace-routes.js, v14-routes.js, api.js, ops/auth-users.js
3	Cas d'utilisation — gestion des véhicules	`pdf/views/use-case-2.mmd`	reference/auth/principal.js, workspace-routes.js, v14-routes.js, api.js, ops/auth-users.js
4	Cas d'utilisation — carte grise	`pdf/views/use-case-3.mmd`	reference/auth/principal.js, workspace-routes.js, v14-routes.js, api.js, ops/auth-users.js
5	Cas d'utilisation — interventions d'entretien et pièces	`pdf/views/use-case-4.mmd`	reference/auth/principal.js, workspace-routes.js, v14-routes.js, api.js, ops/auth-users.js
6	Entités du domaine utilisées par l'application — vue d'ensemble	`diagrams/class-diagram.mmd (+ class-diagram.puml)`	database/schema.sql, database/migrations/*.sql
7	Entités — organisations, comptes et sessions	`pdf/views/class-diagram-1.mmd`	database/schema.sql, database/migrations/*.sql
8	Entités — véhicule, plaques, faits et historique d'identification	`pdf/views/class-diagram-2.mmd`	database/schema.sql, database/migrations/*.sql
9	Entités — carte grise, contact propriétaire et note d'atelier	`pdf/views/class-diagram-3.mmd`	database/schema.sql, database/migrations/*.sql
10	Entités — interventions, opérations, pièces et audit	`pdf/views/class-diagram-4.mmd`	database/schema.sql, database/migrations/*.sql
11	Modules de service et fonctions exportées — vue d'ensemble	`diagrams/class-modules.mmd (copie de 03-class-diagram.md)`	reference/workspace/*.js, documents/document-service.js, vehicle/vehicle-repository.js, workshop/workshop-repository.js, writes.js, db.js
12	Modules — routes de l'espace de travail et services métier	`pdf/views/class-modules-1.mmd`	reference/workspace/*.js, documents/document-service.js, vehicle/vehicle-repository.js, workshop/workshop-repository.js, writes.js, db.js
13	Modules — service des documents et dépôt des véhicules	`pdf/views/class-modules-2.mmd`	reference/workspace/*.js, documents/document-service.js, vehicle/vehicle-repository.js, workshop/workshop-repository.js, writes.js, db.js
14	Modules — dépôt de l'atelier, écritures auditées et accès à la base	`pdf/views/class-modules-3.mmd`	reference/workspace/*.js, documents/document-service.js, vehicle/vehicle-repository.js, workshop/workshop-repository.js, writes.js, db.js
15	Modèle de données — identité, comptes et organisations	`diagrams/er-identity.mmd`	database/schema.sql, database/migrations/*.sql
16	Modèle de données — registre des véhicules, documents et données privées d'atelier (vue d'ensemble)	`diagrams/er-vehicle.mmd`	database/schema.sql, database/migrations/*.sql
17	Modèle de données — véhicules et plaques	`pdf/views/er-vehicle-1.mmd`	database/schema.sql, database/migrations/*.sql

Fig.	Figure	Fichier source	Code ou données de référence
18	Modèle de données — faits et preuves	pdf/views/er-vehicle-2.mmd	database/schema.sql, database/migrations/*.sql
19	Modèle de données — historique, carte grise, contacts et notes par organisation	pdf/views/er-vehicle-3.mmd	database/schema.sql, database/migrations/*.sql
20	Modèle de données — vérifications et événements de service	pdf/views/er-vehicle-4.mmd	database/schema.sql, database/migrations/*.sql
21	Modèle de données — entretien en atelier, pièces et provenance	diagrams/er-workshop.mmd	database/schema.sql, database/migrations/*.sql
22	Modèle de données — capture, réception et revue (vue d'ensemble)	diagrams/er-capture.mmd	database/schema.sql, database/migrations/*.sql
23	Modèle de données — observations, médias et scans de documents	pdf/views/er-capture-1.mmd	database/schema.sql, database/migrations/*.sql
24	Modèle de données — localisations, soumissions, file de revue et consentements	pdf/views/er-capture-2.mmd	database/schema.sql, database/migrations/*.sql
25	Modèle de données — sources de capture, sessions, contributeurs, caméras et mouvements	pdf/views/er-capture-3.mmd	database/schema.sql, database/migrations/*.sql
26	Modèle de données — éléments transverses : audit et limitation de débit	diagrams/er-crosscut.mmd	database/schema.sql, database/migrations/*.sql
27	Architecture logique — couches et chaîne de traitement des requêtes (vue d'ensemble)	diagrams/logical-architecture.mmd	reference/api.js createServer(), reference/workspace|documents|vehicle|workshop|parts|auth, writes.js, db.js, storage.js
28	Architecture logique — couche présentation	pdf/views/logical-architecture-1.mmd	reference/api.js createServer(), reference/workspace|documents|vehicle|workshop|parts|auth, writes.js, db.js, storage.js
29	Architecture logique — chaîne de traitement de l'API, étapes 1 à 5	pdf/views/logical-architecture-2.mmd	reference/api.js createServer(), reference/workspace|documents|vehicle|workshop|parts|auth, writes.js, db.js, storage.js
30	Architecture logique — étapes 6 à 9 et aiguillage vers les modules métier	pdf/views/logical-architecture-3.mmd	reference/api.js createServer(), reference/workspace|documents|vehicle|workshop|parts|auth, writes.js, db.js, storage.js
31	Architecture logique — couche métier	pdf/views/logical-architecture-4.mmd	reference/api.js createServer(), reference/workspace|documents|vehicle|workshop|parts|auth, writes.js, db.js, storage.js
32	Architecture logique — accès aux données, stockage, journaux et adaptateurs non configurés	pdf/views/logical-architecture-5.mmd	reference/api.js createServer(), reference/workspace|documents|vehicle|workshop|parts|auth, writes.js, db.js, storage.js
33	Séquence — enregistrement d'une intervention d'entretien (vue d'ensemble)	diagrams/sequence-intervention.mmd (copie de 04-logical-architecture.md)	workspace-routes.js postIntervention, workspace/intervention-service.js create, writes.js withAudit
34	Séquence — phase 1 : authentification et contrôle des droits (messages 1 à 7)	pdf/views/sequence-intervention-1.mmd	workspace-routes.js postIntervention, workspace/intervention-service.js create, writes.js withAudit
35	Séquence — phase 2 : service métier et transaction (messages 7 à 21)	pdf/views/sequence-intervention-2.mmd	workspace-routes.js postIntervention, workspace/intervention-service.js create, writes.js withAudit
36	Architecture physique — nœuds de production, de pré-production, de développement local et d'intégration continue	diagrams/physical-architecture.mmd	contrôles en lecture seule des serveurs ; .github/workflows/ci.yml ; ops/offhost-backup.js

11.2 Traçabilité consolidée

Élément	Où il a été vérifié
Rôles et droits (technicien 17, gestionnaire 24, administrateur *)	`reference/auth/principal.js`
Authentification : Better Auth, identités par jeton, session propriétaire	`reference/auth/auth.js`, `reference/identity.js`, `reference/session.js`
Ordre de la chaîne de traitement	`reference/api.js`, `createServer()`
Route → droit (141 entrées extraites)	`ALL_ROUTE_SCOPES` dans `api.js`, `workspace-routes.js`, `v14-routes.js`, `v12-routes.js` ; `admin-routes.js` (administrateur seulement)
Entités, attributs, relations	`database/schema.sql`, `database/migrations/*.sql` et le code cité au chapitre « Diagramme de classes »
Modules de service et leurs fonctions	`Object.keys(require(module))` pour chaque module
Statut des tables (utilisée ou schéma seulement)	recherche de chaque nom de table dans `reference/` et `ops/`
Versions, ports, services et base en production	commandes en lecture seule sur le serveur de production
Tables de production = tables du dépôt	`information_schema.tables` de `idauto_production` comparé au schéma : 41 = 41, aucune différence
Intégrations facultatives non configurées	noms de variables comptés dans l'environnement de production : `IDAUTO_TECDOC_*` 0, `IDAUTO_VEHICLE_PROVIDER_*` 0, `IDAUTO_CG_OCR_ENABLED` 0 (valeurs jamais lues)

11.3 Validation des sources

Contrôle	Résultat
Diagramme de classes confronté au SQL : noms et types des attributs, relations contre les clés étrangères appliquées, multiplicités contre la nullabilité	**102 éléments, 0 écart**
Contrôle de structure de toutes les sources Mermaid et PlantUML, et égalité des versions PlantUML et Mermaid pour les cas d'utilisation et les classes	**22 sources, 0 problème**
Modèle de données généré à partir du SQL ; chaque table placée dans exactement un groupe	41 / 41
Rendu Mermaid par GitHub (contrôlé dans un navigateur après publication)	les 11 diagrammes intégrés s'affichent
Recherche de données sensibles dans le dossier (adresses IP, noms d'hôtes privés, chemins personnels, chaînes de connexion, motifs de clés et de jetons)	aucune trouvée

12. Limites, éléments partiels et fonctionnalités prévues

12.1 Éléments partiels

- **Commandes et stock d'atelier.** Les tables (`idauto_workshop_orders`, `idauto_workshop_order_lines`, `idauto_org_stock`) et les fonctions de dépôt existent, mais aucune page de l'application simplifiée ne s'en sert.
- **Catalogue TecDoc.** L'adaptateur est présent mais non configuré en production.
- **Fournisseur de données véhicule (plaque ou VIN).** L'adaptateur est présent mais non configuré, et aucun fournisseur n'est choisi ; la résolution utilise la base locale seulement.

12.2 Fonctionnalités prévues

- **Neuf tables « schéma seulement ».** Elles existent en production mais aucun code ne les utilise : `idauto_camera_sources`, `idauto_capture_sessions`, `idauto_consent_records`, `idauto_document_scans`, `idauto_observation_locations`, `idauto_review_queue`, `idauto_user_roles`, `idauto_vehicle_movements`, `idauto_verifications`.
- **Ancrage blockchain et service de provenance.** Il n'existe que des documents et des fichiers README (`src/*`).
- **Protocole ouvert d'identité des véhicules.** Le répertoire `protocol/` contient des schémas et des vocabulaires (une spécification), pas de code exécuté.
- **Schémas de la plateforme Mythos.** `mythos_core`, `atelier_network` et `fixpert` sont décrits comme cibles dans `docs/ARCHITECTURE.md`, mais ne sont pas implémentés.

12.3 Fonctionnalités abandonnées

- **Lecture automatique (OCR) de la carte grise.** Les routes répondent 410 sauf si `IDAUTO_CG_OCR_ENABLED=1`, et l'interface a été retirée. Le stockage des photos est inchangé.

12.4 Non implémenté

- Réinitialisation du mot de passe en libre-service par courriel : il n'y a pas d'intégration de courriel.
- Intégration Piece.Autos ou Casse.Autos, facturation, gestion de stock : exclues par le propriétaire du projet.

12.5 Informations manquantes et hypothèses

- **Fournisseur du stockage hors serveur.** Il est configuré sur le serveur et n'a pas été lu. Le dossier parle seulement de stockage « compatible S3 ».
- **Calendrier des sauvegardes.** Il appartient à un autre dépôt (`othoth77/mythos-prod`) et peut changer indépendamment de celui-ci.
- **Console d'exploitation (`/admin/*`).** Elle est toujours servie, bien qu'elle soit sortie du menu simplifié ; elle est traitée comme implémentée. Son maintien relève d'une décision produit.
- **Versions de PostgreSQL.** La pré-production (16.9) et la production (15.18) diffèrent d'une version majeure. L'audit n'a pas vérifié si une migration dépend d'une fonction de la version 16.
- **Serveur partagé.** Le serveur de production fait tourner d'autres services, qui consomment aussi son disque et sa mémoire. Ils ne sont pas décrits ici.
- **Rendu PlantUML.** Les fichiers PlantUML (`use-case.puml`, `class-diagram.puml`) n'ont pas pu être rendus : aucun moteur PlantUML n'était disponible (voir l'annexe A). Leur contenu est identique à celui des diagrammes Mermaid inclus dans ce dossier, ce qu'un contrôle automatique a vérifié.

13. Conclusion technique

IDauto, dans sa version en production, est une application volontairement simple. Un seul processus Node.js sans framework web porte les pages, l'API et les règles métier. Il ne dépend que de deux bibliothèques d'exécution, `better-auth` et `pg`, et s'appuie sur une seule base PostgreSQL de 41 tables.

Trois choix structurent l'ensemble :

- **Contrôle d'accès en couches.** Une chaîne de contrôles ordonnée se termine par un contrôle des droits, sur le principe « une route sans droit déclaré est réservée à l'administrateur ». Le cloisonnement par organisation est appliqué une seconde fois dans les requêtes SQL.
- **Traçabilité systématique.** Toute écriture est jumelée à une ligne d'audit dans la même transaction, et les lectures sensibles (contact du propriétaire, VIN) sont elles aussi auditées.
- **Prudence sur les données.** Supprimer revient à archiver, et une intervention annulée est conservée. Les contacts et les notes sont privés à chaque atelier. Les photos ne sont jamais accessibles par une adresse publique.

Sur le plan physique, l'application tourne en une seule instance sur un serveur privé virtuel partagé. Elle n'est exposée que par nginx et sauvegardée chaque jour hors du serveur, avec une vérification quotidienne et un test de restauration mensuel.

Plusieurs parties du dépôt restent hors de cette réalité : des fonctions partielles (commandes, stock, adaptateurs TecDoc et fournisseur de données véhicule non configurés), neuf tables prévues que rien n'utilise, et des documents de plateforme cible antérieurs à la mise en service. Ce dossier les signale comme tels pour qu'aucune ne soit prise pour une fonction en service. Les quatre points ouverts listés au chapitre précédent sont à trancher par le propriétaire du projet : fournisseur de sauvegarde, avenir de la console, alignement des versions de PostgreSQL et rendu PlantUML.

Annexe A — Rendu des diagrammes et validation du document

Chaîne de rendu

- **Mermaid.** Chaque diagramme est rendu par Mermaid 11.17.2 dans Chrome sans interface, en SVG vectoriel. Les libellés sont des textes SVG et non du HTML : ils restent du texte vectoriel dans le PDF. Le thème est noir et blanc, et la police est DejaVu Sans.
- **Modifications au rendu.** Le titre du fichier source devient la légende de la figure. Les émojis des libellés d'acteurs (□, ⚙), que la police d'impression ne sait pas dessiner, sont retirés. Rien d'autre n'est modifié.
- **Vues détaillées.** Elles sont générées par `pdf/views.mjs` à partir des sources, sans réécriture. Un contrôle automatique vérifie que, pour chaque diagramme découpé, l'ensemble des vues contient tous les éléments et toutes les relations de l'original. Résultat : use-case 4 vues, 37 éléments, 39 relations, 0 manquant ; class-diagram 4 vues, 17 éléments, 21 relations, 0 manquant ; class-modules 3 vues, 11 éléments, 21 relations, 0 manquant ; logical-architecture 5 vues, 30 éléments, 38 relations, 0 manquant ; er-vehicle 4 vues, 11 éléments, 23 relations, 0 manquant ; er-capture 3 vues, 12 éléments, 24 relations, 0 manquant ; sequence-intervention 2 vues, 9 éléments, 21 relations, 0 manquant. Une vue peut disposer un groupe verticalement plutôt qu'horizontalement, pour mieux occuper la page ; son contenu est inchangé.
- **PlantUML.** `use-case.puml` et `class-diagram.puml` n'ont **pas été rendus** : aucun moteur PlantUML n'est installé sur la machine de construction, et son téléchargement n'était pas autorisé. Un contrôle automatique a établi que leurs cas d'utilisation, acteurs, relations, classes et multiplicités sont identiques à ceux des diagrammes Mermaid des figures correspondantes.
- **Mise en page.** Chaque figure garde ses proportions et n'est jamais recadrée. Elle reste dans le texte si son plus petit libellé s'imprime à 6 pt ou plus. Sinon, elle occupe une page entière, en portrait ou en paysage selon l'orientation qui l'imprime le plus grand.
- **Seuil de lisibilité.** Il est fixé à 5,5 pt pour le plus petit libellé. Les vues d'ensemble des grands diagrammes passent sous ce seuil : elles montrent la structure, et leurs vues détaillées, toutes au-dessus du seuil, se lisent dans les figures qui les suivent.

Dimensions et lisibilité des figures

Fig.	Fichier	Taille d'origine (px)	Placement	Échelle	Plus petit libellé imprimé
1	`use-case.svg`	1829 × 1502	page portrait	37 %	3,9 pt (vue d'ensemble)
2	`use-case-1.svg`	1173 × 694	texte	58 %	6,1 pt
3	`use-case-2.svg`	1434 × 486	page paysage	71 %	7,4 pt
4	`use-case-3.svg`	1124 × 201	texte	61 %	6,4 pt
5	`use-case-4.svg`	1259 × 413	page paysage	81 %	8,5 pt
6	`class-diagram.svg`	1555 × 2453	page portrait	32 %	2,7 pt (vue d'ensemble)
7	`class-diagram-1.svg`	943 × 1065	page portrait	73 %	6 pt
8	`class-diagram-2.svg`	837 × 1274	page portrait	69 %	5,7 pt
9	`class-diagram-3.svg`	563 × 723	page portrait	100 %	8,3 pt
10	`class-diagram-4.svg`	1401 × 693	page paysage	72 %	6 pt
11	`class-modules.svg`	1349 × 1701	page portrait	48 %	5 pt (vue d'ensemble)
12	`class-modules-1.svg`	1039 × 937	page paysage	64 %	6,7 pt
13	`class-modules-2.svg`	1211 × 777	page paysage	77 %	8,1 pt
14	`class-modules-3.svg`	1396 × 742	page paysage	73 %	7,6 pt
15	`er-identity.svg`	738 × 667	texte	68 %	7,1 pt
16	`er-vehicle.svg`	2589 × 802	page paysage	39 %	4,1 pt (vue d'ensemble)

Fig.	Fichier	Taille d'origine (px)	Placement	Échelle	Plus petit libellé imprimé
17	er-vehicle-1.svg	1657 × 650	page paysage	61 %	6,4 pt
18	er-vehicle-2.svg	458 × 450	texte	100 %	10,5 pt
19	er-vehicle-3.svg	1135 × 328	texte	60 %	6,3 pt
20	er-vehicle-4.svg	741 × 385	texte	92 %	9,7 pt
21	er-workshop.svg	1153 × 1065	page portrait	59 %	6,2 pt
22	er-capture.svg	2369 × 1111	page paysage	43 %	4,5 pt (vue d'ensemble)
23	er-capture-1.svg	1692 × 894	page paysage	60 %	6,3 pt
24	er-capture-2.svg	1127 × 300	texte	61 %	6,4 pt
25	er-capture-3.svg	1216 × 629	page paysage	83 %	8,8 pt
26	er-crosscut.svg	602 × 102	texte	100 %	10,5 pt
27	logical-architecture.svg	2350 × 2170	page portrait	29 %	3,1 pt (vue d'ensemble)
28	logical-architecture-1.svg	826 × 495	texte	83 %	8,7 pt
29	logical-architecture-2.svg	774 × 953	page portrait	88 %	9,3 pt
30	logical-architecture-3.svg	1628 × 821	page paysage	62 %	6,5 pt
31	logical-architecture-4.svg	832 × 1024	page portrait	82 %	8,6 pt
32	logical-architecture-5.svg	1084 × 1045	page portrait	63 %	6,6 pt
33	sequence-intervention.svg	2103 × 1371	page paysage	44 %	3,9 pt (vue d'ensemble)
34	sequence-intervention-1.svg	1199 × 583	page paysage	85 %	7,6 pt
35	sequence-intervention-2.svg	1528 × 918	page paysage	63 %	5,7 pt
36	physical-architecture.svg	1841 × 838	page paysage	55 %	5,8 pt

Validation du PDF

Résultats des contrôles exécutés par `pdf/build-pdf.mjs` sur ce PDF :

Contrôle	Résultat	Détail
Nombre de pages identique entre les deux impressions	**réussi**	69 → 69
Chaque page après la couverture porte « Page n / N » avec son propre numéro	**réussi**	68 pages
Aucune page blanche	**réussi**	
Les numéros de la table des matières sont les pages réelles des titres	**réussi**	60 entries
Chaque titre se trouve sur la page indiquée par la table des matières	**réussi**	
Chaque figure est présente	**réussi**	36 figures
Les figures se suivent dans l'ordre	**réussi**	
Les figures en paysage sont sur des pages en paysage	**réussi**	
Chaque vue détaillée imprime son plus petit libellé à 5,5 pt ou plus	**réussi**	
Aucun débordement horizontal	**réussi**	
Signets du PDF présents	**réussi**	14 top-level entries
Aucune adresse IP (hors 127.0.0.1), chemin personnel, hôte privé, chaîne de connexion ni clé dans le texte	**réussi**	